

HTML Attributes — Complete Notes

1. What are HTML Attributes?

HTML attributes provide **additional information about an HTML element**. They are always written inside the **opening tag** and usually follow the format:

Syntax

```
<tag attribute="value">Content</tag>
```

Example

```
<p id="intro">Hello World</p>
```

Here:

- `<p>` → HTML element
 - `id` → Attribute
 - `"intro"` → Attribute value
 - `Hello World` → Content
-

2. Important Rules of Attributes

Rule 1 — Attributes are written inside the opening tag

```
<h1 class="heading">Welcome</h1>
```

Rule 2 — Most attributes use `name="value"` syntax

```
id="main"  
class="container"  
title="Welcome"
```

Rule 3 — Multiple attributes can be used together

```

```

Rule 4 — Attribute values are normally written inside quotes

```
<p title="My Paragraph">Hello</p>
```

3. id Attribute

The **id** attribute gives a **unique identifier** to an element.

Syntax

```
<tag id="unique-name">Content</tag>
```

Example

```
<h1 id="main-heading">My Website</h1>
```

Uses

- Identifying a specific element
- CSS styling
- JavaScript manipulation
- Creating links to a particular section

Example:

```
<h2 id="about">About Me</h2>
```

```
<a href="#about">Go to About</a>
```

4. class Attribute

The **class** attribute assigns one or more elements to a **class**.

It is commonly used with **CSS** and **JavaScript**.

Syntax

```
<tag class="class-name">Content</tag>
```

Example

```
<p class="text">Hello World</p>
<p class="text">Welcome to HTML</p>
```

The same class can be used on multiple elements.

5. style Attribute

The **style** attribute is used to apply **inline CSS** directly to an element.

Syntax

```
<tag style="property:value;">Content</tag>
```

Example

```
<p style="color: blue;">This is blue text.</p>
```

Multiple CSS properties can also be used:

```
<p style="color: blue; font-size: 20px;">
  Hello World
</p>
```

6. title Attribute

The **title** attribute provides additional information about an element. Browsers commonly display it as a tooltip when the pointer hovers over the element.

Syntax

```
<tag title="information">Content</tag>
```

Example

```
<p title="This is a paragraph">
  Move your pointer over this text.
</p>
```

7. lang Attribute

The **lang** attribute specifies the **language of the document or element**.

Syntax

```
<html lang="language-code">
```

Example

```
<html lang="en">
```

For Hindi:

```
<html lang="hi">
```

It helps browsers, search engines and accessibility tools understand the language.

8. dir Attribute

The **dir** attribute specifies the **text direction**.

Syntax

```
<tag dir="direction">Content</tag>
```

Example

```
<p dir="ltr">Left to right text</p>
```

Common values:

- **ltr** → Left to Right
 - **rtl** → Right to Left
 - **auto** → Browser determines direction
-

9. href Attribute

The **href** attribute specifies the **destination of a hyperlink**.

It is mainly used with the `<a>` element.

Syntax

```
<a href="URL">Link Text</a>
```

Example

```
<a href="https://example.com">Visit Website</a>
```

It can also link to another page:

```
<a href="about.html">About Us</a>
```

10. target Attribute

The **target** attribute specifies **where a linked document should open**.

Example

```
<a href="https://example.com" target="_blank">  
  Open Website  
</a>
```

Common values:

Value	Meaning
<code>_self</code>	Opens in the current browsing context
<code>_blank</code>	Opens in a new browsing context
<code>_parent</code>	Opens in the parent context
<code>_top</code>	Opens in the top-level context

11. **src** Attribute

The **src** attribute specifies the **source/location of external content**.

It is commonly used with `<img>`, `<script>`, `<iframe>`, etc.

Syntax

```

```

Example

```

```

For an image inside a folder:

```

```

12. **alt** Attribute

The **alt** attribute provides **alternative text for an image**.

Syntax

```

```

Example

```

```

It is important for **accessibility** and provides useful text when an image cannot be displayed.

13. **width** and **height** Attributes

These attributes can specify the dimensions of elements such as images.

Example

```

```

Here:

- `width="300"` → Width
 - `height="200"` → Height
-

14. `type` Attribute

The `type` attribute specifies the type of an input or other applicable element.

Example

```
<input type="text">
```

Other common input types:

```
<input type="password">
```

```
<input type="email">
```

```
<input type="number">
```

```
<input type="date">
```

```
<input type="checkbox">
```

```
<input type="radio">
```

```
<input type="file">
```

```
<input type="submit">
```

15. `placeholder` Attribute

The `placeholder` attribute displays a temporary hint inside an input field.

Syntax

```
<input type="text" placeholder="Hint">
```

Example

```
<input type="text" placeholder="Enter your name">
```

16. **name** Attribute

The **name** attribute gives a name to form controls. It is particularly important when form data is submitted.

Example

```
<input type="text" name="username">
```

Another example:

```
<input type="email" name="email">
```

17. **value** Attribute

The **value** attribute specifies an initial or submitted value for certain form controls.

Example

```
<input type="text" name="username" value="Raj">
```

For a button:

```
<input type="submit" value="Login">
```

18. **required** Attribute

required specifies that a form control **must be filled in before submission**.

Example

```
<input type="text" name="username" required>
```

19. disabled Attribute

The **disabled** attribute makes a form control **unavailable for interaction**.

Example

```
<input type="text" value="Disabled Field" disabled>
```

20. readonly Attribute

The **readonly** attribute allows the user to **see the value but not edit it**.

Example

```
<input type="text" value="Raj Gautam" readonly>
```

21. checked Attribute

The **checked** attribute makes a checkbox or radio button selected by default.

Example

```
<input type="checkbox" checked>
```

Radio button:

```
<input type="radio" name="gender" value="male" checked>
```

22. selected Attribute

The **selected** attribute specifies the default selected option in a **<select>** element.

Example

```
<select>
  <option>India</option>
  <option selected>USA</option>
  <option>UK</option>
</select>
```

23. **multiple** Attribute

The **multiple** attribute allows multiple values/selections where supported.

Example

```
<input type="file" multiple>
```

A user can select multiple files.

24. **autofocus** Attribute

The **autofocus** attribute automatically places focus on an applicable form control when the page loads.

Example

```
<input type="text" placeholder="Enter your name" autofocus>
```

25. **min** and **max** Attributes

These attributes specify minimum and maximum allowed values for applicable input controls.

Example

```
<input type="number" min="1" max="100">
```

26. minlength and maxlength

These attributes specify the minimum and maximum number of characters allowed.

Example

```
<input type="text" minlength="3" maxlength="20">
```

27. Boolean Attributes

Some HTML attributes are **Boolean attributes**. Their presence represents the true state.

Examples

```
<input type="text" required>
```

```
<input type="checkbox" checked>
```

```
<input type="text" disabled>
```

```
<input type="text" readonly>
```

```
<input type="file" multiple>
```

Common Boolean attributes:

- `required`
 - `disabled`
 - `readonly`
 - `checked`
 - `selected`
 - `multiple`
 - `autofocus`
-

28. data-* Attributes

HTML allows developers to store **custom data** on elements using `data-*` attributes.

Syntax

```
<tag data-name="value">Content</tag>
```

Example

```
<button data-user-id="101">View Profile</button>
```

JavaScript can later read this custom data.

29. `aria-*` Attributes

`aria-*` attributes provide additional information for **accessibility technologies**, such as screen readers.

Example

```
<button aria-label="Close menu">X</button>
```

These should be used appropriately; normal semantic HTML should be preferred whenever possible.

30. Important Attributes at a Glance

Attribute	Main Use
<code>id</code>	Unique identification
<code>class</code>	Grouping elements
<code>style</code>	Inline CSS
<code>title</code>	Additional information/tooltip
<code>lang</code>	Document language
<code>href</code>	Link destination

<code>target</code>	Link opening context
<code>src</code>	External resource location
<code>alt</code>	Alternative image text
<code>width</code>	Width
<code>height</code>	Height
<code>type</code>	Input/control type
<code>name</code>	Form-control name
<code>value</code>	Initial/control value
<code>placeholder</code>	Input hint
<code>required</code>	Required field
<code>disabled</code>	Disable control
<code>readonly</code>	Prevent editing
<code>checked</code>	Default checkbox/radio selection
<code>selected</code>	Default option
<code>multiple</code>	Allow multiple selections
<code>autofocus</code>	Automatically focus control
<code>min</code>	Minimum value
<code>max</code>	Maximum value
<code>minlength</code>	Minimum characters
<code>maxlength</code>	Maximum characters
<code>data-*</code>	Custom data
<code>aria-*</code>	Accessibility information

31. Practical Program Using HTML Attributes

Below is a small **Student Profile + Registration Form** that combines many of the attributes discussed above.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Profile</title>
</head>

<body>

  <header id="top">
    <h1 class="heading" title="Student Profile">
      Student Profile
    </h1>
  </header>

  <main>

    <!-- Profile Section -->
    <section id="profile">

      <h2 class="section-title">My Profile</h2>

      <p class="description">
        Hello! I am a BCA student learning HTML and Web Development.
      </p>

      <a
        href="https://github.com/"
        target="_blank"
        title="Visit GitHub"
      >
        Visit GitHub
      </a>

    </section>
```

```
<!-- Registration Form -->
<section id="registration">
```

```
  <h2 class="section-title">Student Registration</h2>
```

```
  <form>
```

```
    <label for="name">Name:</label>
```

```
    <input
```

```
      type="text"
```

```
      id="name"
```

```
      name="username"
```

```
      placeholder="Enter your name"
```

```
      minlength="3"
```

```
      maxlength="30"
```

```
      required
```

```
      autofocus
```

```
    >
```

```
    <br><br>
```

```
    <label for="email">Email:</label>
```

```
    <input
```

```
      type="email"
```

```
      id="email"
```

```
      name="email"
```

```
      placeholder="Enter your email"
```

```
      required
```

```
    >
```

```
    <br><br>
```

```
    <label for="age">Age:</label>
```

```
    <input
```

```
      type="number"
```

```
      id="age"
```

```
      name="age"
```

```
      min="15"
```

```
      max="100"
```

```
      required
```

```
    >
```

```
    <br><br>
```

```
    <label for="course">Course:</label>
```

```
<select id="course" name="course">

    <option value="bca" selected>BCA</option>
    <option value="btech">B.Tech</option>
    <option value="bsc">B.Sc</option>

</select>

<br><br>

<label>
    <input
        type="checkbox"
        name="terms"
        value="accepted"
        required
    >
    I agree to the terms and conditions.
</label>

<br><br>

<input
    type="submit"
    value="Register"
>

</form>

</section>

</main>

<footer>

    <p
        class="footer-text"
        title="Copyright information"
    >
        © 2026 Student Profile
    </p>

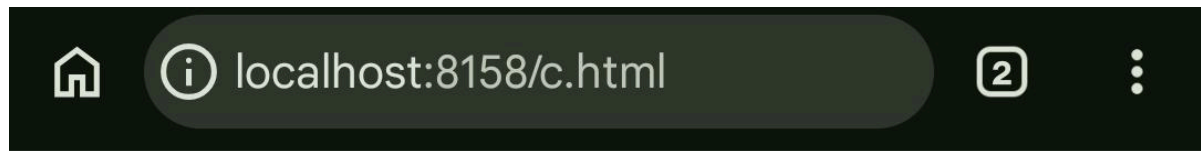
    <a href="#top">Back to Top</a>

</footer>

</body>
```

</html>

O.p



Student Profile

My Profile

 Student Profile Photo

Hello! I am a BCA student learning HTML and Web Development.

[Visit GitHub](#)

Student Registration

Name:

Email:

Age:

Course:

☐ I agree to the terms and conditions.

© 2026 Student Profile

[Back to Top](#)

Attributes used in this program

The program demonstrates:

lang, id, class, title, src, alt, width, height, href, target, type, name, placeholder, minlength, maxlength, required, autofocus, min, max, value, and selected.

Exam definition to remember:

HTML attributes are special properties used inside the opening tag of an HTML element to provide additional information or modify the element's behavior.

HTML Formatting — Complete Notes

1. What is HTML Formatting?

HTML formatting means using HTML elements to change or define the appearance, importance, emphasis, or presentation of text on a webpage.

HTML provides several formatting elements such as `<b>`, `<strong>`, `<i>`, `<em>`, `<u>`, `<mark>`, `<small>`, `<del>`, `<ins>`, `<sub>`, and `<sup>`.

2. Bold Text — `<b>`

The `<b>` element displays text in bold without adding any special semantic importance.

Syntax

```
<b>Text</b>
```

Example

```
<p>This is <b>important text</b>.</p>
```

Output:

This is important text.

3. Strong Text — `<strong>`

The `<strong>` element indicates that the text has strong importance.

Browsers normally display it in bold.

Syntax

```
<strong>Important Text</strong>
```

Example

```
<p><strong>Warning:</strong> Do not share your password.</p>
```

`<b>` vs `<strong>`

- `<b>` → visual bold formatting
- `<strong>` → indicates strong importance

For meaningful/important content, `<strong>` is generally preferred.

4. Italic Text — `<i>`

The `<i>` element represents text in an alternate voice or mood, such as a technical term, foreign word, or phrase.

Browsers normally display it in italic.

Syntax

```
<i>Text</i>
```

Example

```
<p>HTML is an <i>interesting</i> language.</p>
```

5. Emphasized Text — `<em>`

The `<em>` element represents emphasis.

Browsers normally display it in italic.

Syntax

```
<em>Text</em>
```

Example

```
<p>You <em>must</em> complete this assignment.</p>
```

`<i>` vs `<em>`

- `<i>` → alternate voice/mood or specialized text
- `<em>` → semantic emphasis

6. Underlined Text — `<u>`

The `<u>` element represents text with an unarticulated annotation, and browsers normally underline it.

Syntax

```
<u>Text</u>
```

Example

```
<p>This is <u>underlined text</u>.</p>
```

For purely decorative underlining, CSS is usually a better choice.

7. Highlighted Text — `<mark>`

The `<mark>` element represents text that is highlighted or marked for reference.

Syntax

```
<mark>Text</mark>
```

Example

```
<p>HTML is <mark>very important</mark> for web development.</p>
```

8. Smaller Text — `<small>`

The `<small>` element represents side comments or small print, such as copyright information.

Syntax

```
<small>Text</small>
```

Example

```
<p><small>© 2026 My Website</small></p>
```

9. Deleted Text — `<del>`

The `<del>` element represents text that has been deleted from a document.

Browsers normally display it with a strikethrough.

Syntax

```
<del>Old Text</del>
```

Example

```
<p>Old Price: <del>₹999</del></p>
```

10. Inserted Text — `<ins>`

The `<ins>` element represents text that has been inserted into a document.

Browsers normally display it underlined.

Syntax

```
<ins>New Text</ins>
```

Example

```
<p>New Price: <ins>₹799</ins></p>
```

Combined Example

```
<p>
```

```
  Old Price: <del>₹999</del>
```

```
  New Price: <ins>₹799</ins>
```

```
</p>
```

11. Subscript — `<sub>`

The `<sub>` element displays text below the normal baseline.

It is commonly used for chemical formulas.

Syntax

`<sub>Text</sub>`

Example

`<p>Water formula: H<sub>2</sub>O</p>`

Output: H₂O

12. Superscript — `<sup>`

The `<sup>` element displays text above the normal baseline.

It is commonly used for mathematical powers and references.

Syntax

`<sup>Text</sup>`

Example

`<p>Mathematics: 10<sup>2</sup> = 100</p>`

Output: 10² = 100

13. Code Text — `<code>`

The `<code>` element represents a short piece of computer code.

Browsers normally display it in a monospace font.

Syntax

```
<code>code</code>
```

Example

```
<p>Use <code>print()</code> to display output in Python.</p>
```

14. Preformatted Text — `<pre>`

The `<pre>` element preserves spaces and line breaks exactly as written.

Syntax

```
<pre>
```

Text

Text

Text

```
</pre>
```

Example

```
<pre>
```

Name: Raj

Course: BCA

Subject: HTML

</pre>

15. Quotation — <q>

The <q> element represents a short inline quotation.

Example

```
<p>He said, <q>HTML is easy to learn.</q></p>
```

The browser typically adds quotation marks automatically.

16. Block Quotation — <blockquote>

The <blockquote> element represents a long quotation from another source.

Example

```
<blockquote>
```

Learning web development requires practice and consistency.

```
</blockquote>
```

17. Abbreviation — <abbr>

The <abbr> element represents an abbreviation or acronym.

The **title** attribute can provide the full meaning.

Example

<p>

<abbr title="HyperText Markup Language">HTML</abbr>

is used to create webpages.

</p>

When the user hovers over **HTML**, the full meaning can be shown.

18. Line Break —

The
 element creates a line break.

It is a void element, so it does not have a closing tag.

Syntax

Example

<p>

Hello Raj

Welcome to HTML.

</p>

19. Horizontal Rule — <hr>

The <hr> element represents a thematic break between sections.

Example

`<h2>HTML</h2>`

`<hr>`

`<h2>CSS</h2>`

20. Formatting Elements — Quick Reference

Element	Purpose
<code></code>	Bold/attention without added importance
<code></code>	Strong importance
<code><i></code>	Alternate voice/mood or specialized text
<code></code>	Emphasis
<code><u></code>	Underlined/annotated text
<code><mark></code>	Highlighted text

<code><small></code>	Small print/side comments
----------------------------	---------------------------

<code></code>	Deleted text
--------------------------	--------------

<code><ins></code>	Inserted text
--------------------------	---------------

<code><sub></code>	Subscript
--------------------------	-----------

<code><sup></code>	Superscript
--------------------------	-------------

<code><code></code>	Computer code
---------------------------	---------------

<code><pre></code>	Preserves formatting
--------------------------	----------------------

<code><q></code>	Short quotation
------------------------	-----------------

<code><blockquote></code>	Long quotation
---------------------------------	----------------

<code><abbr></code>	Abbreviation
---------------------------	--------------

<code>
</code>	Line break
-------------------------	------------

<code><hr></code>	Thematic break
-------------------------	----------------

21. Important Difference: **** vs ****

Bold Text

Important Text

Both normally appear bold, but their meaning is different.

- **** → mainly draws attention visually.
 - **** → tells browsers and assistive technologies that the content is important.
-

22. Important Difference: **<i>** vs ****

<i>Technical term</i>

Very important

- **<i>** → alternate voice, mood, term, etc.
 - **** → semantic emphasis.
-

23. Practical Program Using HTML Formatting

This program combines many formatting elements into a small HTML Learning Page.

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>HTML Formatting</title>

</head>

<body>

<h1>HTML Formatting Examples</h1>

<hr>

<h2>Text Formatting</h2>

<p>

This is bold text.

</p>

<p>

This is important text.

</p>

<p>

This is <i>italic text</i>.

</p>

<p>

This is emphasized text.

</p>

<p>

This is <u>underlined text</u>.

</p>

<p>

This is <mark>highlighted text</mark>.

</p>

<p>

This is <small>small text</small>.

</p>

<hr>

<h2>Price Example</h2>

<p>

Old Price:

₹999

</p>

<p>

New Price:

<ins>₹799</ins>

</p>

<hr>

<h2>Mathematics</h2>

<p>

10² = 100

</p>

<p>

Water = H₂O

</p>

<hr>

<h2>Programming</h2>

<p>

Python output function:

<code>print()</code>

</p>

<pre>

Name: Raj

Course: BCA

Subject: HTML

Topic: Formatting

</pre>

<hr>

<h2>Quotation</h2>

<p>

He said,

<q>Practice makes programming easier.</q>

</p>

<blockquote>

HTML provides the structure of a webpage.

CSS controls its presentation.

JavaScript adds behavior and interaction.

</blockquote>

<hr>

<p>

<abbr title="HyperText Markup Language">HTML</abbr>

is the standard markup language for creating webpages.

</p>

</body>

</html>

Programming

Python output function: `print()`

Name: Raj
Course: BCA
Subject: HTML
Topic: Formatting

Quotation

He said, "Practice makes programming easier."

HTML provides the structure of a webpage.
CSS controls its presentation. JavaScript
adds behavior and interaction.

HTML is the standard markup language for creating webpages.

HTML Formatting Examples

Text Formatting

This is **bold text**.

This is **important text**.

This is *italic text*.

This is *emphasized text*.

This is underlined text.

This is highlighted text.

This is small text.

Price Example

Old Price: ~~₹999~~

New Price: ₹799

Mathematics

$$10^2 = 100$$

Water = H₂O

Exam Definition

HTML formatting is the process of using HTML elements to define the appearance, emphasis, importance, and presentation of text and other content on a webpage.

Key point: Modern HTML generally favors semantic elements such as `<strong>`, `<em>`, `<mark>`, `<del>`, and `<ins>` when they accurately describe the meaning of the content. CSS should be used for purely visual styling.

HTML Style Attribute — Complete Notes

1. What is the `style` Attribute?

The `style` attribute is used to apply CSS styles directly to an HTML element. It is called inline CSS.

It can change properties such as:

- Text color
 - Background color
 - Font size
 - Font family
 - Text alignment
 - Width and height
 - Margin and padding
 - Borders
 - And many more
-

2. Syntax

```
<tag style="property: value;">Content</tag>
```

Example

```
<p style="color: blue;">Hello World</p>
```

Here:

- **style** → HTML attribute
 - **color** → CSS property
 - **blue** → CSS value
-

3. Changing Text Color

The **color** property changes the text color.

Example

```
<p style="color: red;">This is red text.</p>
```

Another example:

```
<h1 style="color: blue;">Welcome to HTML</h1>
```

4. Changing Background Color

Use **background-color**.

Syntax

```
<tag style="background-color: color;">
```

Example

```
<p style="background-color: yellow;">
```

This paragraph has a yellow background.

```
</p>
```

5. Changing Font Size

Use the **font-size** property.

Example

```
<p style="font-size: 25px;">
```

Large Text

```
</p>
```

You can also use other CSS units such as **em**, **rem**, **%**, etc.

6. Changing Font Family

Use the **font-family** property.

Example

```
<p style="font-family: Arial;">
```

This text uses Arial.

```
</p>
```

Multiple fallback fonts can be specified:

```
<p style="font-family: Arial, sans-serif;">
```

Hello World

```
</p>
```

7. Text Alignment

Use the `text-align` property.

Example

```
<h1 style="text-align: center;">
```

Welcome

```
</h1>
```

Common values:

left

center

right

justify

8. Making Text Bold

You can use the CSS `font-weight` property.

Example

```
<p style="font-weight: bold;">
```

Bold Text

```
</p>
```

9. Making Text Italic

Use **font-style**.

Example

```
<p style="font-style: italic;">
```

Italic Text

```
</p>
```

10. Adding a Border

Use the **border** property.

Example

```
<p style="border: 2px solid black;">
```

Text with border

```
</p>
```

The basic syntax is:

border: width style color;

Example:

```
<div style="border: 2px solid blue;">
```

My Box

```
</div>
```

11. Setting Width and Height

Example

```
<div style="width: 300px; height: 150px;">
```

My Box

```
</div>
```

12. Adding Padding

padding creates space inside an element, between its content and border.

Example

```
<div style="padding: 20px; border: 1px solid black;">
```

Content

```
</div>
```

13. Adding Margin

margin creates space outside an element.

Example

```
<div style="margin: 20px;">
```

Content

```
</div>
```

14. Using Multiple CSS Properties

Multiple properties are separated using a semicolon (;).

Syntax

```
<tag style="property1: value1; property2: value2;">
```

Example

```
<p style="color: white; background-color: black; font-size: 20px;">
```

Styled Text

```
</p>
```

Here three CSS properties are used:

1. `color`
 2. `background-color`
 3. `font-size`
-

15. `style` Attribute with `<div>`

The `style` attribute is commonly used with `<div>` to create simple boxes.

Example

```
<div style="background-color: lightblue; padding: 20px; border: 2px solid blue;">
```

```
<h2>Student Profile</h2>
```

```
<p>Name: Raj</p>
```

```
<p>Course: BCA</p>
```

```
</div>
```

16. Important CSS Properties for **style**

CSS Property	Use	Example
<code>color</code>	Text color	<code>color: red;</code>
<code>background-color</code>	Background color	<code>background-color: yellow;</code>
<code>font-size</code>	Text size	<code>font-size: 20px;</code>
<code>font-family</code>	Font	<code>font-family: Arial;</code>
<code>font-weight</code>	Text thickness	<code>font-weight: bold;</code>
<code>font-style</code>	Italic style	<code>font-style: italic;</code>
<code>text-align</code>	Text alignment	<code>text-align: center;</code>
<code>border</code>	Border	<code>border: 1px solid black;</code>
<code>width</code>	Width	<code>width: 300px;</code>
<code>height</code>	Height	<code>height: 150px;</code>
<code>margin</code>	Outside spacing	<code>margin: 20px;</code>

padding

Inside spacing

padding: 20px;

17. Complete Example

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>HTML Style Attribute</title>
```

```
</head>
```

```
<body>
```

```
  <h1 style="color: blue; text-align: center;">
```

```
    HTML Style Attribute
```

```
</h1>
```

```
  <p style="color: green; font-size: 20px;">
```

```
    This is a styled paragraph.
```

```
</p>
```

```
<div
```

```
  style="
```

```
    background-color: lightblue;
```

```
width: 300px;
padding: 20px;
margin: 20px;
border: 2px solid blue;
text-align: center;
"
>
```

```
<h2 style="color: darkblue;">
```

Student Profile

```
</h2>
```

```
<p style="font-weight: bold;">
```

Name: Raj

```
</p>
```

```
<p>
```

Course: BCA

```
</p>
```

```
</div>
```

```
</body>
```

```
</html>
```

HTML Style Attribute

This is a styled paragraph.



What this program demonstrates

- `color`
- `background-color`
- `font-size`
- `font-weight`
- `text-align`
- `width`
- `padding`
- `margin`
- `border`

18. Advantages of `style` Attribute

- Easy to use for small changes.

- Useful for testing CSS quickly.
- No separate CSS file is required.
- Good for applying a style to one specific element.

19. Disadvantages

- Difficult to maintain in large websites.
- The same styles may need to be repeated.
- Makes HTML code longer.
- Separating HTML and CSS is generally better for larger projects.

For larger websites, prefer external CSS:

```
<link rel="stylesheet" href="style.css">
```

Exam Definition

The HTML **style** attribute is used to apply inline CSS styles directly to an HTML element. It contains one or more CSS declarations in the form **property: value;**.

Most Important Syntax

```
<tag style="property: value;">Content</tag>
```

Remember

HTML → Structure

CSS → Styling

style attribute → CSS directly inside an HTML element

HTML Colors and Sizes — Complete Notes

HTML itself provides the structure of a webpage, while colors and sizes are mainly controlled using CSS. For basic HTML practice, these styles can be applied using the **style** attribute.

1. HTML Colors

Colors can be applied to text, backgrounds, borders, and other elements using CSS properties.

color Property

The **color** property changes the text color.

Syntax

```
<tag style="color: value;">Content</tag>
```

Example

```
<p style="color: red;">This is red text.</p>
```

background-color

The **background-color** property changes the background color of an element.

Syntax

```
<tag style="background-color: value;">Content</tag>
```

Example

```
<p style="background-color: yellow;">
```

Yellow background

</p>

2. Different Ways to Specify Colors

CSS supports several ways of writing colors.

A. Color Names

You can use predefined color names.

```
<p style="color: red;">Red Text</p>
```

```
<p style="color: blue;">Blue Text</p>
```

```
<p style="color: green;">Green Text</p>
```

```
<p style="color: purple;">Purple Text</p>
```

Examples:

red, blue, green, yellow, black, white, orange, purple, gray

B. HEX Color

HEX colors use a # followed by six hexadecimal digits.

Syntax

```
color: #RRGGBB;
```

Example

`<p style="color: #ff0000;">Red Text</p>`

Some common HEX colors:

Color	HEX
Black	<code>#000000</code>
White	<code>#ffffff</code>
Red	<code>#ff0000</code>
Green	<code>#008000</code>
Blue	<code>#0000ff</code>
Yellow	<code>#ffff00</code>

C. RGB Color

RGB stands for Red, Green, Blue.

Syntax

`color: rgb(red, green, blue);`

Each value normally ranges from 0 to 255.

Example

`<p style="color: rgb(255, 0, 0);">`

Red Text

</p>

Another example:

<p style="color: rgb(0, 0, 255);">

Blue Text

</p>

D. RGBA Color

RGBA adds an alpha channel, which controls transparency.

Syntax

color: rgba(red, green, blue, alpha);

Example

<p style="color: rgba(255, 0, 0, 0.5);">

Semi-transparent red

</p>

The alpha value ranges from 0 to 1:

- 0 → fully transparent
- 1 → fully opaque

E. HSL Color

HSL stands for:

- H → Hue
- S → Saturation
- L → Lightness

Syntax

color: hsl(hue, saturation, lightness);

Example

```
<p style="color: hsl(0, 100%, 50%);">
```

Red Text

```
</p>
```

3. HTML Text Size

Text size is controlled using the CSS **font-size** property.

Syntax

```
<tag style="font-size: value;">Content</tag>
```

Example

```
<p style="font-size: 20px;">
```

This is 20px text.

```
</p>
```

4. Common Font Size Units

px — Pixels

```
<p style="font-size: 24px;">24px Text</p>
```

px is a fixed CSS length unit.

% — Percentage

```
<p style="font-size: 120%;">Larger Text</p>
```

The size is calculated relative to the relevant parent/default size.

em

em is relative to the font size of the element's parent.

```
<p style="font-size: 1.5em;">
```

Large Text

```
</p>
```

rem

rem is relative to the root (**html**) element's font size.

```
<p style="font-size: 2rem;">
```

Large Text

```
</p>
```

5. Width and Height

The **width** and **height** properties control the dimensions of elements.

Syntax

```
<tag style="width: value; height: value;">
```

Example

```
<div style="width: 300px; height: 150px;">
```

My Box

```
</div>
```

6. Different Size Units

Common CSS length units include:

Unit	Meaning	Example
px	Pixels	width: 300px;
%	Percentage	width: 80%;
em	Relative to parent font size	font-size: 1.5em;
rem	Relative to root font size	font-size: 2rem;

vw

Viewport width

width: 50vw;

vh

Viewport height

height: 50vh;

7. HTML Headings and Size

HTML provides six heading levels:

`<h1>Heading 1</h1>`

`<h2>Heading 2</h2>`

`<h3>Heading 3</h3>`

`<h4>Heading 4</h4>`

`<h5>Heading 5</h5>`

`<h6>Heading 6</h6>`

<h1> represents the highest-level heading and **<h6>** the lowest-level heading.

Example

`<h1>My Website</h1>`

`<h2>About Me</h2>`

`<h3>My Skills</h3>`

Important: Don't choose heading elements just because of their default visual size. Use them according to the document structure, and use CSS to control their appearance.

8. Combining Color and Size

You can use multiple CSS properties in one **style** attribute.

Example

```
<p style="color: blue; font-size: 25px;">
```

Large Blue Text

```
</p>
```

Another example:

```
<h1 style="
```

```
color: white;
```

```
background-color: blue;
```

```
font-size: 40px;
```

```
text-align: center;
```

```
">
```

Welcome

```
</h1>
```

9. Complete Practical Program

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>HTML Colors and Sizes</title>
```

```
</head>
```

```
<body>
```

```
<h1 style="
    color: white;
    background-color: #2563eb;
    font-size: 40px;
    text-align: center;
">
```

HTML Colors & Sizes

```
</h1>
```

```
<p style="
    color: red;
    font-size: 22px;
">
    This is red and 22px text.
</p>
```

```
<p style="
    color: rgb(0, 128, 0);
    font-size: 20px;
">
    This is green text.
</p>
```

```
<div style="
    width: 300px;
    height: 150px;
    background-color: #facc15;
    border: 2px solid #000000;
```

padding: 20px;

">

<h2 style="color: #111827;">

Student Profile

</h2>

<p style="font-size: 18px;">

Name: Raj

</p>

<p style="font-size: 16px;">

Course: BCA

</p>

</div>

</body>

</html>

HTML Colors & Sizes

This is red and 22px text.

This is green text.

Student Profile

Name: Raj

Course: BCA

Quick Revision

Color

color: red;

background-color: blue;

Font Size

font-size: 20px;

Width & Height

`width: 300px;`

`height: 200px;`

HEX

`color: #ff0000;`

RGB

`color: rgb(255, 0, 0);`

Multiple Styles

`<p style="color: blue; font-size: 20px;">`

Hello World

`</p>`

Exam Definition

HTML colors and sizes are primarily controlled using CSS properties such as **color**, **background-color**, **font-size**, **width**, and **height**, which can be applied inline using the HTML **style** attribute.

HTML Tables — Complete Notes

1. What is an HTML Table?

An HTML table is used to display data in a structured format of rows and columns.

Tables are useful for displaying:

- Student records
- Marksheets
- Product lists

- Employee information
- Timetables
- Prices
- Reports

A table is created using the `<table>` element.

2. Basic Table Structure

The main elements used in an HTML table are:

Element	Purpose
<code><table></code>	Creates the table
<code><tr></code>	Creates a table row
<code><th></code>	Creates a table header cell
<code><td></code>	Creates a table data cell
<code><caption></code>	Adds a table title/caption
<code><thead></code>	Groups the header rows
<code><tbody></code>	Groups the main data rows
<code><tfoot></code>	Groups footer/summary rows

3. Basic Syntax

```
<table>

  <tr>

    <th>Header 1</th>

    <th>Header 2</th>

  </tr>

  <tr>

    <td>Data 1</td>

    <td>Data 2</td>

  </tr>

</table>
```

4. **<table>** Element

The **<table>** element defines the table itself.

Syntax

```
<table>

  ...

</table>
```

Example

```
<table>

  <tr>
```

```
<td>Raj</td>
<td>BCA</td>
</tr>
</table>
```

5. **<tr>** — Table Row

<tr> stands for Table Row.

It creates a horizontal row in a table.

Syntax

```
<tr>
...
</tr>
```

Example

```
<table>
<tr>
  <td>Raj</td>
  <td>BCA</td>
</tr>
</table>
```

Here **<tr>** creates one row.

6. `<th>` — Table Header

`<th>` stands for Table Header.

It defines a header cell.

Browsers normally display header cells as bold and centered by default.

Example

```
<table>

  <tr>

    <th>Name</th>

    <th>Course</th>

    <th>Semester</th>

  </tr>

</table>
```

7. `<td>` — Table Data

`<td>` stands for Table Data.

It represents a normal data cell.

Example

```
<table>

  <tr>

    <td>Raj</td>

    <td>BCA</td>

    <td>5th</td>

  </tr>
```

</table>

8. Complete Basic Table

```
<table>
```

```
  <tr>
```

```
    <th>Name</th>
```

```
    <th>Course</th>
```

```
    <th>Marks</th>
```

```
  </tr>
```

```
  <tr>
```

```
    <td>Raj</td>
```

```
    <td>BCA</td>
```

```
    <td>85</td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td>Aman</td>
```

```
    <td>BCA</td>
```

```
    <td>78</td>
```

```
  </tr>
```

```
</table>
```

Conceptually, it looks like:

Name	Course	Marks
Raj	BCA	85
Aman	BCA	78

9. border — Table Border

You can use CSS to add borders.

Example

```
<table style="border: 1px solid black;">
```

However, to create borders around individual cells, CSS is generally better:

```
<table style="border-collapse: collapse;">
```

```
<tr>
```

```
<th style="border: 1px solid black;">Name</th>
```

```
<th style="border: 1px solid black;">Marks</th>
```

```
</tr>
```

```
<tr>
```

```
<td style="border: 1px solid black;">Raj</td>
```

```
<td style="border: 1px solid black;">85</td>
```

```
</tr>
```

</table>

10. border-collapse

The **border-collapse** CSS property controls how table borders are displayed.

Example

```
<table style="border-collapse: collapse;">
```

collapse combines adjacent cell borders into a single border.

11. caption

The **<caption>** element adds a title/caption to the table.

Syntax

```
<table>
```

```
  <caption>Table Title</caption>
```

```
  ...
```

```
</table>
```

Example

```
<table>
```

```
<caption>Student Records</caption>
```

```
<tr>
```

```
  <th>Name</th>
```

```
  <th>Course</th>
```

```
</tr>
```

```
<tr>
```

```
  <td>Raj</td>
```

```
  <td>BCA</td>
```

```
</tr>
```

```
</table>
```

12. <thead>

<thead> groups the header rows of a table.

Example

```
<table>
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Name</th>
```

```
      <th>Course</th>
```

```
    </tr>
```

```
</thead>
```

```
</table>
```

13. <tbody>

<tbody> groups the main data rows.

Example

```
<table>
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Name</th>
```

```
      <th>Course</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    <tr>
```

```
      <td>Raj</td>
```

```
      <td>BCA</td>
```

```
    </tr>
```

```
    <tr>
```

```
      <td>Aman</td>
```

```
        <td>BCA</td>

    </tr>

</tbody>

</table>
```

14. <tfoot>

<tfoot> groups the footer or summary rows.

Example

```
<table>

    <thead>

        <tr>

            <th>Subject</th>

            <th>Marks</th>

        </tr>

    </thead>

    <tbody>

        <tr>

            <td>HTML</td>

            <td>85</td>

        </tr>
```

```
<tr>
  <td>CSS</td>
  <td>80</td>
</tr>
</tbody>

<tfoot>
  <tr>
    <th>Total</th>
    <th>165</th>
  </tr>
</tfoot>

</table>
```

15. colspan

colspan allows one cell to span multiple columns.

Syntax

```
<td colspan="number">Content</td>
```

Example

```
<table>
```

```
<tr>
```

```
<th colspan="3">Student Information</th>
</tr>

<tr>
  <td>Raj</td>
  <td>BCA</td>
  <td>5th Semester</td>
</tr>

</table>
```

Here the header cell occupies 3 columns.

16. rowspan

rowspan allows one cell to span multiple rows.

Syntax

```
<td rowspan="number">Content</td>
```

Example

```
<table>

<tr>
  <th>Name</th>
  <th>Subject</th>
  <th>Marks</th>
```

```
</tr>
```

```
<tr>
```

```
  <td rowspan="2">Raj</td>
```

```
  <td>HTML</td>
```

```
  <td>85</td>
```

```
</tr>
```

```
<tr>
```

```
  <td>CSS</td>
```

```
  <td>80</td>
```

```
</tr>
```

```
</table>
```

Raj occupies two rows.

17. Styling an HTML Table

CSS can be used to make a table easier to read.

Example

```
<table style="
```

```
  border-collapse: collapse;
```

```
  width: 100%;
```

```
<tr>
```

```
  <th style="
```

```
    border: 1px solid black;
```

```
    padding: 10px;
```

```
">
```

```
    Name
```

```
</th>
```

```
  <th style="
```

```
    border: 1px solid black;
```

```
    padding: 10px;
```

```
">
```

```
    Course
```

```
</th>
```

```
</tr>
```

```
<tr>
```

```
  <td style="
```

```
    border: 1px solid black;
```

```
    padding: 10px;
```

```
">
```

```
    Raj
```

```
</td>
```

```
  <td style="
```

```
    border: 1px solid black;
```

```
padding: 10px;

">

    BCA

</td>

</tr>

</table>
```

18. Important Table Attributes vs CSS

You may see older HTML examples using attributes such as:

```
<table border="1">
```

This works in some contexts but **border** as a table HTML attribute is obsolete in modern HTML.

Prefer CSS:

```
<table style="border: 1px solid black;">
```

For larger projects, use a separate CSS file.

19. Complete Practical Program

Here is a BCA Student Marks Table using the major table elements.

```
<!DOCTYPE html>

<html lang="en">

<head>
```

```
<meta charset="UTF-8">
```

```
<meta
```

```
    name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
```

```
>
```

```
<title>BCA Student Marks</title>
```

```
</head>
```

```
<body>
```

```
    <h1>BCA Student Marks</h1>
```

```
    <table style="
```

```
        width: 100%;
```

```
        border-collapse: collapse;
```

```
">
```

```
        <caption>
```

```
            BCA Semester Marks
```

```
        </caption>
```

```
        <thead>
```

```
            <tr>
```

```
                <th style="border: 1px solid black; padding: 10px;">
```

```
                    Roll No.
```

```
                </th>
```

```
                <th style="border: 1px solid black; padding: 10px;">
```

```
                    Name
```

```
                </th>
```

```
                <th style="border: 1px solid black; padding: 10px;">
```

```
                    Subject
```

```
</th>

<th style="border: 1px solid black; padding: 10px;">
    Marks
</th>
</tr>
</thead>
<tbody>
<tr>
    <td style="border: 1px solid black; padding: 10px;">
        101
    </td>
    <td style="border: 1px solid black; padding: 10px;">
        Raj
    </td>
    <td style="border: 1px solid black; padding: 10px;">
        HTML
    </td>
    <td style="border: 1px solid black; padding: 10px;">
        85
    </td>
</tr>
<tr>
    <td style="border: 1px solid black; padding: 10px;">
        102
    </td>
    <td style="border: 1px solid black; padding: 10px;">
```

Aman

</td>

<td style="border: 1px solid black; padding: 10px;">

HTML

</td>

<td style="border: 1px solid black; padding: 10px;">

78

</td>

</tr>

<tr>

<td style="border: 1px solid black; padding: 10px;">

103

</td>

<td style="border: 1px solid black; padding: 10px;">

Ravi

</td>

<td style="border: 1px solid black; padding: 10px;">

HTML

</td>

<td style="border: 1px solid black; padding: 10px;">

91

</td>

</tr>

</tbody>

<tfoot>

<tr>

```
<th
    colspan="3"
    style="border: 1px solid black; padding: 10px;"
>
    Highest Marks
</th>
<th style="border: 1px solid black; padding: 10px;">
    91
</th>
</tr>
</tfoot>
</table>
</body>
</html>
```

BCA Student Marks

BCA Semester Marks

Roll No.	Name	Subject	Marks
101	Raj	HTML	85
102	Aman	HTML	78
103	Ravi	HTML	91
Highest Marks			91

20. Quick Revision

Basic Table

```
<table>
```

```
  <tr>
```

```
    <th>Name</th>
```

```
    <th>Marks</th>
```

```
  </tr>
```

```
  <tr>
```

```
    <td>Raj</td>
```

```
<td>85</td>

</tr>

</table>
```

Main Elements

<table> → Table

<tr> → Row

<th> → Header cell

<td> → Data cell

<caption> → Table title

<thead> → Header section

<tbody> → Main data section

<tfoot> → Footer/summary section

Cell Merging

```
<td colspan="2">Two Columns</td>
```

```
<td rowspan="2">Two Rows</td>
```

Exam Definition

An HTML table is used to organize and present data in rows and columns using elements such as **<table>**, **<tr>**, **<th>**, and **<td>**.

HTML **id** Attribute — Complete Notes

1. What is the **id** Attribute?

The **id** attribute is used to give an HTML element a unique identifier.

An **id** is especially useful when you want to:

- Identify a specific HTML element
- Apply CSS to a specific element
- Access an element using JavaScript
- Create links to a specific section of a webpage
- Work with forms and labels

Syntax

```
<tag id="unique-name">Content</tag>
```

Example

```
<p id="intro">Welcome to my website.</p>
```

Here:

- **p** → HTML element
 - **id** → Attribute
 - **"intro"** → ID value
-

2. Basic Example

```
<h1 id="main-heading">My Website</h1>
```

```
<p id="description">
```

This is my first webpage.

```
</p>
```

Each element has a different **id**.

3. **id** Should Be Unique

An **id** is intended to identify one specific element within a document.

Good Example

```
<h1 id="title">My Website</h1>
```

```
<p id="description">Welcome to my website.</p>
```

Avoid

```
<h1 id="title">My Website</h1>
```

```
<p id="title">Welcome</p>
```

The same **id** should not be assigned to multiple elements.

If you need to identify multiple elements as a group, use the **class** attribute.

4. **id** with CSS

An **id** can be selected in CSS using the **#** symbol.

HTML

```
<h1 id="heading">Welcome</h1>
```

CSS

```
#heading {  
    color: blue;
```

```
font-size: 35px;  
}
```

The `#heading` selector targets the element whose `id` is `heading`.

5. `id` with JavaScript

JavaScript can find an element by its `id`.

HTML

```
<p id="message">Hello World</p>
```

JavaScript

```
const message = document.getElementById("message");
```

```
message.textContent = "Hello Rajl!";
```

The `getElementById()` method searches for an element with the specified `id`.

6. `id` with Links

An `id` can be used to create a link that jumps to a specific section of the same webpage.

Syntax

```
<a href="#id-name">Go to Section</a>
```

Example

```
<a href="#about">Go to About Section</a>
```

```
<h2 id="about">About Me</h2>
```

When the link is clicked, the browser navigates to the element with `id="about"`.

7. `id` with Different Sections

A webpage can have different sections with unique IDs.

```
<nav>
```

```
  <a href="#home">Home</a>
```

```
  <a href="#about">About</a>
```

```
  <a href="#contact">Contact</a>
```

```
</nav>
```

```
<section id="home">
```

```
  <h2>Home</h2>
```

```
</section>
```

```
<section id="about">
```

```
  <h2>About</h2>
```

```
</section>
```

```
<section id="contact">

  <h2>Contact</h2>

</section>
```

This is commonly used for single-page websites.

8. **id** with **<label>** in Forms

The **id** attribute is particularly useful with form labels.

Example

```
<label for="username">Username:</label>
```

```
<input
  type="text"
  id="username"
  name="username"
>
```

Here:

label's for="username"



input's id="username"

The **for** value connects the label to the corresponding input's **id**.

9. Rules for Good IDs

Use meaningful and descriptive IDs.

Good

```
<div id="student-profile">
```

```
<section id="about">
```

```
<input id="username">
```

Avoid

```
<div id="x123">
```

if the ID has no meaningful purpose.

Prefer:

```
<div id="profile">
```

10. ID Naming Examples

Common naming styles include:

Hyphen style

```
<div id="student-profile">
```

Underscore style

```
<div id="student_profile">
```

Camel case

```
<div id="studentProfile">
```

For HTML/CSS projects, lowercase names with hyphens are a common readable convention:

```
<section id="student-profile">
```

11. id vs class

This is an important concept for exams and coding.

id	class
Identifies a specific element	Groups elements
Intended to be unique in a document	Can be reused
CSS selector uses #	CSS selector uses .
Example: <code>id="header"</code>	Example: <code>class="card"</code>

HTML

```
<h1 id="main-title">My Website</h1>
```

```
<p class="text">Paragraph 1</p>
```

```
<p class="text">Paragraph 2</p>
```

CSS

```
#main-title {  
    color: blue;  
}
```

```
.text {  
    font-size: 18px;  
}
```

12. Important Points

1. **id** is a global HTML attribute, so it can be used on most HTML elements.
2. An **id** should be unique within the document.
3. The value should be meaningful and descriptive.
4. **id** is commonly used with CSS.
5. **id** is commonly used with JavaScript.
6. **id** can create internal page navigation.
7. **id** is useful for connecting **<label>** elements with form controls.
8. CSS uses **#** to select an ID.

13. Complete Practical Program

This example demonstrates **id** with navigation, CSS, JavaScript, and a form.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta
```

```
    name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
```

```
>
```

```
<title>HTML ID Attribute</title>
```

```
<style>
```

```
    #header {
```

```
        text-align: center;
```

```
        background-color: lightblue;
```

```
        padding: 20px;
```

```
    }
```

```
    #about {
```

```
        padding: 30px;
```

```
    }
```

```
    #contact {
```

```
        padding: 30px;
```

```
    }
```

```
    #message {
```

```
        color: green;
```

```
        font-weight: bold;
```

```
    }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
    <header id="header">
```

<h1>My Student Website</h1>

<nav>

About |

Contact

</nav>

</header>

<section id="about">

<h2>About Me</h2>

<p>

I am a BCA student learning HTML.

</p>

<p id="message">

Welcome to my website!

</p>

</section>

<section id="contact">

<h2>Contact</h2>

<form>

<label for="username">

Name:

</label>

<input

type="text"

id="username"

name="username"

placeholder="Enter your name"

```
>

<br><br>

<button

    type="button"

    onclick="showMessage()"

>

    Click Me

</button>

</form>

</section>

<script>

    function showMessage() {

        document.getElementById("message").textContent =

            "Button clicked successfully!";

    }

</script>

</body>

</html>
```

My Student Website

[About](#) | [Contact](#)

About Me

I am a BCA student learning HTML.

Welcome to my website!

Contact

Name:

Click Me

In this program, **id** is used for:

header → Identify the header

about → Navigate to About section

contact → Navigate to Contact section

message → Change text using JavaScript

username → Connect label with input

Quick Revision

Syntax

```
<tag id="unique-name">Content</tag>
```

CSS

```
#unique-name {  
    color: red;  
}
```

JavaScript

```
document.getElementById("unique-name");
```

Internal Link

```
<a href="#about">About</a>
```

```
<section id="about">
```

```
    <h2>About</h2>
```

```
</section>
```

Exam Definition

The HTML **id** attribute is a global attribute used to assign a unique identifier to an HTML element. It can be used for CSS styling, JavaScript manipulation, internal navigation, and connecting form labels with controls.

HTML Layout and Responsive Web Design — Complete Notes

1. What is HTML Layout?

HTML Layout refers to the way different parts of a webpage are organized and arranged.

A typical webpage may contain:

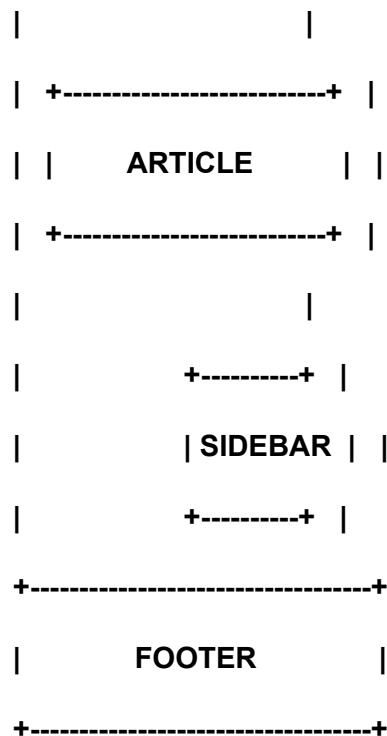
- Header
- Navigation
- Main content
- Sections
- Articles
- Sidebar
- Footer

HTML provides the structure, while CSS is mainly used to control the layout, spacing, alignment, and visual appearance.

2. Basic HTML Layout

A common webpage structure looks like this:

```
+-----+
|      HEADER      |
+-----+
|      NAVIGATION  |
+-----+
|                   |
|      MAIN CONTENT      |
```



3. Important HTML Layout Elements

Modern HTML provides semantic elements that describe the purpose of different parts of a webpage.

Element	Purpose
<header>	Introductory/header content
<nav>	Navigation links
<main>	Main content of the page
<section>	Thematic section

<code><article></code>	Independent/self-contained content
<code><aside></code>	Related content/sidebar
<code><footer></code>	Footer information
<code><div></code>	Generic container

4. `<header>`

The `<header>` element represents introductory content for a page or section.

Example

```
<header>
```

```
  <h1>My Website</h1>
```

```
  <p>Welcome to my website</p>
```

```
</header>
```

A header may contain:

- Logo
 - Website title
 - Introduction
 - Navigation
-

5. <nav>

The **<nav>** element contains important navigation links.

Example

```
<nav>
```

```
  <a href="#home">Home</a>
```

```
  <a href="#about">About</a>
```

```
  <a href="#services">Services</a>
```

```
  <a href="#contact">Contact</a>
```

```
</nav>
```

6. <main>

The **<main>** element contains the main content of the webpage.

A document normally has one primary **<main>** element.

Example

```
<main>
```

```
  <h1>Welcome to My Website</h1>
```

```
  <p>
```

```
    This is the main content of the webpage.
```

```
  </p>
```

</main>

7. <section>

<section> represents a thematic section of content.

Example

```
<section id="about">
```

```
  <h2>About Me</h2>
```

```
  <p>
```

```
    I am a BCA student learning web development.
```

```
  </p>
```

```
</section>
```

A section should generally have a heading.

8. <article>

The <article> element represents self-contained content that could potentially stand on its own.

Examples:

- Blog post

- News article
- Forum post
- Product review

Example

`<article>`

`<h2>Introduction to HTML</h2>`

`<p>`

HTML is the standard markup language
for creating webpages.

`</p>`

`</article>`

9. `<aside>`

`<aside>` represents content that is related to the main content, such as a sidebar.

Example

`<aside>`

`<h3>Related Topics</h3>`

`<ul>`

`<li>CSS</li>`

`<li>JavaScript</li>`

```
<li>Web Development</li>
```

```
</ul>
```

```
</aside>
```

10. `<footer>`

The `<footer>` element represents footer information for a page or section.

Example

```
<footer>
```

```
<p>© 2026 My Website</p>
```

```
<a href="#top">Back to Top</a>
```

```
</footer>
```

11. `<div>` for Layout

`<div>` is a generic container.

It has no special semantic meaning by itself, but it is widely used with CSS to create layouts.

Example

```
<div class="container">
```

```
<div class="box">
```

Content 1

```
</div>
```

```
<div class="box">
```

Content 2

```
</div>
```

```
</div>
```

CSS can then control the position and appearance of these boxes.

12. Semantic Layout Example

A modern webpage can be structured like this:

```
<body>
```

```
<header>
```

Website Header

```
</header>
```

```
<nav>
```

Navigation

```
</nav>
```

```
<main>
```

```
<section>
```

Main Section

```
<article>

    Article Content

</article>

</section>

<aside>

    Sidebar

</aside>

</main>

<footer>

    Website Footer

</footer>

</body>
```

This makes the document structure easier for developers, browsers, search engines, and assistive technologies to understand.

13. What is Responsive Web Design?

Responsive Web Design means creating a website that adapts to different screen sizes and devices.

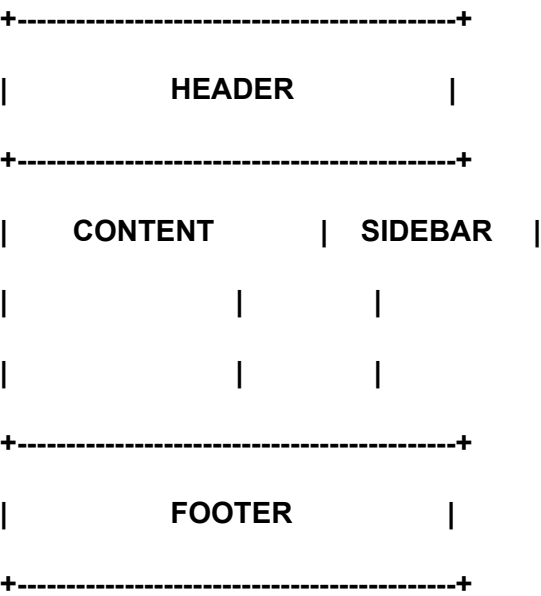
A responsive website should work well on:

-  Mobile phones
-  Tablets
-  Laptops
-  Desktop monitors

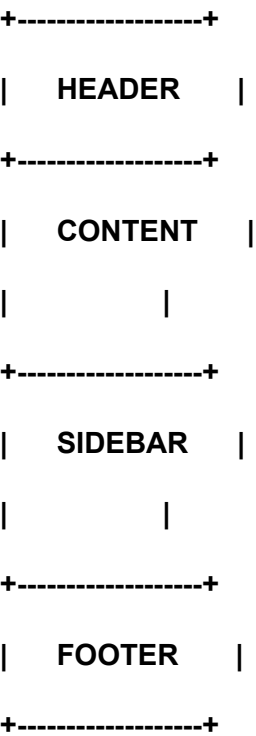
Instead of creating completely separate websites for every device, the layout adjusts according to the available screen size.

14. Responsive Design Idea

Desktop



Mobile



The important idea is that the layout changes according to the available screen width.

15. Viewport Meta Tag

One of the most important things for responsive webpages is the viewport meta tag.

Syntax

```
<meta  
  name="viewport"  
  content="width=device-width, initial-scale=1.0"  
>
```

Example

```
<head>  
  
  <meta  
    name="viewport"  
    content="width=device-width, initial-scale=1.0"  
  >  
  
</head>
```

Meaning

width=device-width

→ Makes the page layout width correspond to the device's viewport width.

initial-scale=1.0

→ Sets the initial zoom level.

16. Responsive Images

Images should generally be allowed to shrink within their container.

CSS

```
img {  
    max-width: 100%;  
    height: auto;  
}
```

This helps prevent an image from overflowing its container on small screens.

Example

```

```

17. Flexible Width

Instead of always using fixed widths such as:

```
width: 1000px;
```

responsive layouts often use flexible values:

```
width: 100%;
```

`max-width: 1000px;`

This allows the element to shrink on smaller screens while limiting its maximum width on larger screens.

18. CSS Flexbox

Flexbox is one of the most useful CSS layout systems for responsive designs.

Example

```
.container {  
    display: flex;  
    gap: 20px;  
}
```

HTML:

```
<div class="container">  
  
    <div>Box 1</div>  
  
    <div>Box 2</div>  
  
    <div>Box 3</div>  
  
</div>
```

The boxes can be arranged flexibly.

19. Flexbox for Mobile

You can change the direction on smaller screens.

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

```
@media (max-width: 600px) {
```

```
  .container {  
    flex-direction: column;  
  }
```

```
}
```

On larger screens:

Box 1 | Box 2 | Box 3

On smaller screens:

Box 1

Box 2

Box 3

20. Media Queries

A media query allows CSS rules to change depending on characteristics such as viewport width.

Syntax

```
@media (condition) {
```

```
    /* CSS rules */
```

```
}
```

Example

```
@media (max-width: 600px) {
```

```
    body {
```

```
        font-size: 16px;
```

```
    }
```

```
}
```

This rule applies when the viewport width is 600px or less.

21. Responsive Navigation Example

```
nav {
```

```
    display: flex;
```

```
    gap: 20px;
```

```
}
```

```
@media (max-width: 600px) {
```

```
nav {  
    flex-direction: column;  
}  
  
}
```

Desktop:

Home | About | Services | Contact

Mobile:

Home

About

Services

Contact

22. Responsive Units

Useful CSS units for responsive layouts include:

Unit	Meaning
px	Pixel
%	Percentage of a relevant reference size

em

Relative to the element's font size

rem

Relative to the root element's font size

vw

Viewport width

vh

Viewport height

Example:

```
.container {  
  width: 90%;  
  max-width: 1200px;  
}
```

23. Complete Responsive HTML Program

This example combines HTML layout + semantic elements + Flexbox + responsive design.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta
```

```
    name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
```

```
>
```

<title>Responsive Student Website</title>

<style>

*** {**

box-sizing: border-box;

}

body {

margin: 0;

font-family: Arial, sans-serif;

}

header {

padding: 25px;

text-align: center;

background-color: #222;

color: white;

}

nav {

display: flex;

justify-content: center;

gap: 25px;

padding: 15px;

background-color: #ddd;

}

nav a {

text-decoration: none;

}

main {

```
display: flex;

gap: 20px;

max-width: 1200px;

margin: auto;

padding: 20px;
}

section {

    flex: 3;
}

aside {

    flex: 1;

    padding: 20px;

    background-color: #eee;
}

article {

    padding: 20px;

    border: 1px solid #ccc;
}

img {

    max-width: 100%;

    height: auto;
}

footer {

    padding: 20px;

    text-align: center;

    background-color: #222;
```

```
        color: white;
    }

    /* Responsive Design */

    @media (max-width: 700px) {
        main {
            flex-direction: column;
        }

        nav {
            flex-direction: column;
            text-align: center;
            gap: 10px;
        }
    }
</style>
</head>
<body>
    <header>
        <h1>My Student Website</h1>
        <p>BCA Student | Web Development Learner</p>
    </header>
    <nav>
        <a href="#home">Home</a>
        <a href="#about">About</a>
        <a href="#skills">Skills</a>
        <a href="#contact">Contact</a>
```

</nav>

<main>

<section id="home">

<article>

<h2>Welcome</h2>

<p>

Welcome to my responsive HTML website.

</p>

<img

src="student.jpg"

alt="Student image"

>

</article>

<article id="about">

<h2>About Me</h2>

<p>

I am learning HTML, CSS and JavaScript

for web development.

</p>

</article>

</section>

<aside id="skills">

<h2>My Skills</h2>

HTML

CSS

JavaScript

Python

</aside>

</main>

<footer id="contact">

<p>© 2026 My Student Website</p>

</footer>

</body>

</html>

My Student Website

BCA Student | Web Development Learner

[Home](#)

[About](#)

[Skills](#)

[Contact](#)

Welcome

Welcome to my responsive HTML website.

 Student image

About Me

I am learning HTML, CSS and JavaScript for web development.

My Skills

- HTML
- CSS
- JavaScript
- Python

© 2026 My Student Website

24. What You Learn from This Program

This single program demonstrates:

HTML Structure

`<html>`

`<head>`

`<body>`

`<header>`

`<nav>`

`<main>`

`<section>`

`<article>`

`<aside>`

`<footer>`

CSS Layout

`display: flex;`

`flex-direction;`

`gap;`

`max-width;`

`margin;`

`padding;`

Responsive Design

`<meta name="viewport">`

`@media`

`max-width`

`flex-direction: column`

25. Important Points for Semester Exam

HTML Layout

HTML layout is the structural organization of webpage content into different areas such as header, navigation, main content, sections, articles, sidebars, and footer.

Responsive Web Design

Responsive web design is an approach to designing webpages so that their layout and content adapt appropriately to different screen sizes and devices.

Remember

HTML → Structure

CSS → Layout + Styling

Flexbox/Grid → Modern Layout Systems

Media Queries → Adapt layout for different conditions

Viewport Meta Tag → Important for mobile rendering

Responsive Design → One webpage that adapts to different screen sizes

HTML Semantic Tags — Complete Notes

1. What are Semantic Tags?

Semantic HTML tags are elements whose names clearly describe the meaning and purpose of their content.

For example:

```
<header>Website Header</header>
```

From the tag name, we can understand that the content is a header.

Semantic HTML helps:

- Developers understand the code
 - Search engines understand page structure
 - Screen readers and other assistive technologies interpret content
 - Maintain and organize large websites more easily
-

2. Semantic vs Non-Semantic Elements

Semantic Element

```
<article>
```

```
  <h2>HTML Tutorial</h2>
```

```
  <p>Learn HTML step by step.</p>
```

```
</article>
```

<article> tells us that the content is an independent article.

Non-Semantic Element

```
<div>

  <h2>HTML Tutorial</h2>

  <p>Learn HTML step by step.</p>

</div>
```

<div> is a generic container. It does not tell us what the content means.

3. Important Semantic Tags

Tag	Purpose
<header>	Header/introduction for a page or section
<nav>	Navigation links
<main>	Main content
<section>	Thematic section
<article>	Independent/self-contained content
<aside>	Related content/sidebar

<code><footer></code>	Footer information
<code><figure></code>	Self-contained media/content
<code><figcaption></code>	Caption for a figure
<code><details></code>	Expandable details
<code><summary></code>	Visible heading for <code><details></code>
<code><time></code>	Date/time
<code><address></code>	Contact information
<code><mark></code>	Highlighted/relevant text

4. `<header>`

The `<header>` element represents introductory content for a page or section.

It can contain:

- Website logo
- Website title
- Introduction
- Navigation

Syntax

`<header>`

Content

`</header>`

Example

`<header>`

`<h1>My Website</h1>`

`<p>Welcome to my website.</p>`

`</header>`

5. `<nav>`

The `<nav>` element contains important navigation links.

Syntax

`<nav>`

Navigation Links

`</nav>`

Example

`<nav>`

`<a href="#home">Home</a>`

```
<a href="#about">About</a>
```

```
<a href="#contact">Contact</a>
```

```
</nav>
```

6. `<main>`

The `<main>` element represents the primary content of the document.

Example

```
<main>
```

```
<h1>HTML Tutorial</h1>
```

```
<p>
```

This is the main content of the webpage.

```
</p>
```

```
</main>
```

A page generally has one primary `<main>` element.

7. `<section>`

The `<section>` element represents a thematic grouping of content.

A section generally has a heading.

Example

```
<section id="about">
```

```
  <h2>About Me</h2>
```

```
  <p>
```

```
    I am learning web development.
```

```
  </p>
```

```
</section>
```

Common uses:

- About section
 - Services section
 - Skills section
 - Contact section
-

8. **<article>**

The **<article>** element represents self-contained content that could potentially be distributed or reused independently.

Examples:

- Blog post
- News article
- Product review
- Forum post

Example

```
<article>
```

`<h2>What is HTML?</h2>`

`<p>`

HTML is the standard markup language
for creating webpages.

`</p>`

`</article>`

9. `<aside>`

The `<aside>` element represents content that is related to the surrounding content, often a sidebar.

Example

`<aside>`

`<h3>Related Topics</h3>`

`<ul>`

`<li>CSS</li>`

`<li>JavaScript</li>`

`<li>Web Development</li>`

`</ul>`

`</aside>`

10. `<footer>`

The `<footer>` element represents footer information for a page or section.

It can contain:

- Copyright information
- Contact information
- Related links
- Author information

Example

`<footer>`

`<p>© 2026 My Website</p>`

`</footer>`

11. `<figure>`

The `<figure>` element represents self-contained content, such as:

- Images
- Diagrams
- Charts
- Illustrations
- Code examples

Example

`<figure>`

```

```

```
</figure>
```

12. <figcaption>

<figcaption> provides a caption or description for a <figure>.

Example

```
<figure>  
  
    
  
  <figcaption>  
    HTML Logo  
  </figcaption>  
  
</figure>
```

13. <details>

The `<details>` element creates an expandable section.

Example

```
<details>

  <summary>What is HTML?</summary>

  <p>

    HTML stands for HyperText Markup Language.

  </p>

</details>
```

The user can click the summary to show or hide the additional content.

14. <summary>

`<summary>` provides the visible heading for a `<details>` element.

Example

```
<details>

  <summary>Click to learn more</summary>

  <p>

    This content can be expanded and collapsed.

  </p>

</details>
```

15. <time>

The **<time>** element represents a specific date or time.

Example

<p>

Exam Date:

<time datetime="2026-08-20">

August 20, 2026

</time>

</p>

The **datetime** attribute provides a machine-readable value.

16. <address>

The **<address>** element represents contact information for the relevant page or article.

Example

<address>

Email:

example@example.com

</address>

It is not intended for arbitrary postal addresses that aren't contact information.

17. `<mark>`

The `<mark>` element represents text that has been highlighted because it is relevant.

Example

```
<p>
```

```
    HTML is a
```

```
    <mark>markup language</mark>.
```

```
</p>
```

18. Semantic Text Elements

Some semantic elements are also used for text-level meaning.

`<strong>`

Indicates strong importance.

```
<strong>Important Notice</strong>
```

`<em>`

Indicates emphasis.

```
<em>Very important</em>
```

`<abbr>`

Represents an abbreviation.

```
<abbr title="HyperText Markup Language">
```

HTML

```
</abbr>
```

<cite>

Represents the title of a creative work.

```
<cite>Introduction to HTML</cite>
```

19. Semantic Page Structure

A typical semantic webpage can be structured like this:

```
<header>
  Header
</header>
|
<nav>
  Navigation
</nav>
|
<main>
  |
  +-----+-----+
  |           |
<section>    <aside>
```

Content

Sidebar

|

<article>

Article

</article>

|

</main>

|

<footer>

Footer

</footer>

20. Complete Semantic HTML Program

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta
```

```
    name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
```

```
>
```

```
  <title>Semantic HTML Website</title>
```

```
</head>
```

```
<body>
```

```
<!-- Header -->

<header id="top">

  <h1>My Learning Website</h1>

  <p>

    Learn HTML step by step.

  </p>

</header>

<!-- Navigation -->

<nav>

  <a href="#home">Home</a> |

  <a href="#articles">Articles</a> |

  <a href="#about">About</a>

</nav>

<!-- Main Content -->

<main>

  <section id="home">

    <h2>Welcome</h2>

    <p>

      Welcome to my HTML learning website.

    </p>

  </section>

  <!-- Articles -->

  <section id="articles">

    <h2>Latest Articles</h2>

    <article>

      <h3>What is HTML?</h3>
```

<p>

HTML is the standard markup language
used to structure webpages.

</p>

<time datetime="2026-08-17">

August 17, 2026

</time>

</article>

<article>

<h3>What is CSS?</h3>

<p>

CSS is used to style and lay out
HTML documents.

</p>

</article>

</section>

<!-- About -->

<section id="about">

<h2>About</h2>

<figure>

<figcaption>

HTML Logo

</figcaption>

</figure>

</section>

<!-- Sidebar -->

<aside>

<h3>Related Topics</h3>

HTML

CSS

JavaScript

</aside>

<!-- Details -->

<details>

<summary>

What does HTML stand for?

</summary>

<p>

HTML stands for

HyperText Markup Language.

</p>

</details>

</main>

<!-- Footer -->

<footer>

<p>

© 2026 My Learning Website

</p>

<address>

Contact:

example@example.com

</address>

</footer>

</body>

</html>

My Learning Website

Learn HTML step by step.

[Home](#) | [Articles](#) | [About](#)

Welcome

Welcome to my HTML learning website.

Latest Articles

What is HTML?

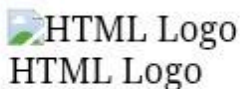
HTML is the standard markup language used to structure webpages.

August 17, 2026

What is CSS?

CSS is used to style and lay out HTML documents.

About



Related Topics

- [HTML](#)
- [CSS](#)
- [JavaScript](#)

► [What does HTML stand for?](#)

© 2026 My Learning Website

Contact: example@example.com

21. Semantic Tags vs **<div>**

Non-semantic approach

```
<div id="header">
```

Header

```
</div>
```

```
<div id="navigation">
```

Navigation

```
</div>
```

```
<div id="content">
```

Content

```
</div>
```

```
<div id="footer">
```

Footer

```
</div>
```

Semantic approach

```
<header>
```

Header

```
</header>
```

```
<nav>
```

Navigation

`</nav>`

`<main>`

Content

`</main>`

`<footer>`

Footer

`</footer>`

The second version communicates the meaning and structure much more clearly.

22. Important Points for Semester

1. Semantic tags clearly describe the meaning of their content.
 2. They make HTML easier to read and maintain.
 3. They can improve accessibility.
 4. They help search engines understand the structure of a webpage.
 5. `<div>` and `<span>` are generic, non-semantic containers.
 6. Semantic HTML does not replace CSS; CSS is still used for visual styling.
 7. Use semantic elements according to their meaning rather than their default appearance.
-

Quick Revision

`<header>` → Header

`<nav>` → Navigation

`<main>` → Main content

`<section>` → Thematic section

`<article>` → Independent content

<aside> → Related/sidebar content

<footer> → Footer

<figure> → Self-contained media/content

<figcaption> → Figure caption

<details> → Expandable content

<summary> → Details heading

<time> → Date/time

<address> → Contact information

<mark> → Highlighted text

**** → Strong importance

**** → Emphasis

<abbr> → Abbreviation

Exam Definition

Semantic HTML tags are HTML elements that clearly describe the meaning and purpose of the content they contain, such as **<header>**, **<nav>**, **<main>**, **<section>**, **<article>**, **<aside>**, and **<footer>**.

HTML **<head>** Part — Complete Notes

1. What is the **<head>** Element?

The **<head>** element contains information and resources about an HTML document that are generally not displayed as the main page content.

It is placed between **<html>** and **<body>**.

Basic Structure

```
<!DOCTYPE html>

<html>

<head>

    <!-- Metadata and resources -->

</head>

<body>

    <!-- Visible webpage content -->

</body>

</html>
```

2. Basic Syntax

```
<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>My Website</title>

</head>
```

3. Important Elements Used Inside `<head>`

Element	Purpose
<code><title></code>	Sets the page title
<code><meta></code>	Provides metadata
<code><link></code>	Connects external resources
<code><style></code>	Contains internal CSS
<code><script></code>	Includes or loads JavaScript
<code><base></code>	Sets a base URL for relative URLs

4. `<title>`

The `<title>` element defines the title of the HTML document.

It is commonly shown in:

- Browser tab
- Bookmarks
- Search engine results

Syntax

```
<title>Page Title</title>
```

Example

```
<head>
```

```
  <title>My Portfolio</title>
```

```
</head>
```

5. `<meta>`

`<meta>` provides metadata about the document.

Metadata is information about the webpage that isn't normally displayed as page content.

A. Character Encoding

Use:

```
<meta charset="UTF-8">
```

UTF-8 supports a very large range of characters and is the standard choice for most modern HTML documents.

Example

```
<head>
```

```
  <meta charset="UTF-8">
```

```
<title>My Website</title>
```

```
</head>
```

6. Viewport Meta Tag

This is particularly important for responsive websites.

Syntax

```
<meta  
  name="viewport"  
  content="width=device-width, initial-scale=1.0"  
>
```

Meaning

width=device-width



Use the device's viewport width

initial-scale=1.0



Initial zoom level = 1

Without an appropriate viewport configuration, mobile browsers may render pages in a way that makes responsive layouts behave unexpectedly.

7. Meta Description

A description can provide a summary of the page.

Example

```
<meta  
  name="description"  
  content="Learn HTML, CSS and JavaScript."  
>
```

This is metadata that can be used by search engines when generating search-result snippets, although search engines decide what snippet to display.

8. Meta Keywords

You may see old HTML tutorials using:

```
<meta  
  name="keywords"  
  content="HTML, CSS, JavaScript"  
>
```

However, major search engines generally do not use the **keywords** meta tag for ranking. It is not something you need for modern HTML development.

9. **<link>**

The **<link>** element connects the HTML document to an external resource.

The most common use is connecting an external CSS file.

Syntax

```
<link rel="stylesheet" href="style.css">
```

Example

```
<head>
```

```
  <title>My Website</title>
```

```
  <link
```

```
    rel="stylesheet"
```

```
    href="style.css"
```

```
  >
```

```
</head>
```

10. Favicon

A favicon is the small icon associated with a webpage, often shown in the browser tab.

Example

```
<link
```

```
  rel="icon"
```

```
  href="favicon.png"
```

```
>
```

You can also specify a MIME type when useful:

```
<link
  rel="icon"
  type="image/png"
  href="favicon.png"
>
```

11. <style>

The <style> element contains internal CSS.

Example

```
<head>

  <style>

    body {

      font-family: Arial, sans-serif;

    }

    h1 {

      color: blue;

    }

  </style>
```

```
</head>
```

This is different from the **style** attribute:

Inline CSS

```
<h1 style="color: blue;">
```

Hello

```
</h1>
```

Internal CSS

```
<style>
```

```
h1 {  
  color: blue;  
}
```

```
</style>
```

External CSS

```
<link rel="stylesheet" href="style.css">
```

For larger projects, external CSS is generally easier to maintain.

12. **<script>**

The `<script>` element is used to include or embed JavaScript.

Example:

```
<script src="script.js"></script>
```

You can also put JavaScript directly inside it:

```
<script>
```

```
    console.log("Hello World");
```

```
</script>
```

For scripts that can be loaded without blocking HTML parsing, `defer` is commonly useful when loading an external script:

```
<script src="script.js" defer></script>
```

13. `<base>`

The `<base>` element specifies the base URL used for relative URLs in the document.

Example

```
<base href="https://example.com/">
```

Then:

```
<a href="about.html">About</a>
```

can resolve relative to that base URL.

`<base>` is less commonly needed in basic HTML projects, but it is part of the `<head>` knowledge.

14. Complete `<head>` Example

```
<!DOCTYPE html>

<html lang="en">

<head>

  <!-- Character Encoding -->

  <meta charset="UTF-8">

  <!-- Responsive Design -->

  <meta

    name="viewport"

    content="width=device-width, initial-scale=1.0"

  >

  <!-- Page Title -->

  <title>Raj's Portfolio</title>

  <!-- Page Description -->

  <meta

    name="description"

    content="Personal portfolio website"

  >

  <!-- Favicon -->

  <link

    rel="icon"

    type="image/png"
```

```
        href="favicon.png"
    >
    <!-- External CSS -->
    <link
        rel="stylesheet"
        href="style.css"
    >
    <!-- Internal CSS -->
    <style>
        body {
            margin: 0;
        }
    </style>
    <!-- JavaScript -->
    <script
        src="script.js"
        defer
    ></script>
</head>
<body>
    <h1>My Portfolio</h1>
</body>
</html>
```

15. <head> vs <body>

This is important for exams.

<head>	<body>
Contains document metadata and resources	Contains page content
Usually not directly visible	Visible webpage content
Contains <title>	Contains headings, paragraphs, images, etc.
Contains <meta>	Contains <p>, <h1>, , etc.
Contains <link>	Contains links such as <a>
Can contain <style>	Can contain visible content
Can contain <script>	Can also contain scripts

Simple way to remember

<head>

Information ABOUT the webpage

</head>

<body>

Content OF the webpage

</body>

16. Complete HTML Document Structure

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta
```

```
    name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
```

```
>
```

```
<title>My Website</title>
```

```
<link
```

```
  rel="stylesheet"
```

```
  href="style.css"
```

```
>
```

```
</head>
```

```
<body>
```

```
  <header>
```

```
    <h1>My Website</h1>
```

```
  </header>
```

```
<main>
```

```
  <section>
```

```
    <h2>Welcome</h2>
```

```
<p>
    This is my webpage.
</p>
</section>
</main>
<footer>
    <p>© 2026 My Website</p>
</footer>
</body>
</html>
```

Ot;-

My Website

Welcome

This is my webpage.

© 2026 My Website

Quick Revision

<head>



Document metadata + resources

<title> → Page title

<meta> → Metadata

<link> → External resources

<style> → Internal CSS

<script> → JavaScript

<base> → Base URL

Most Important Syntax

<head>

<meta charset="UTF-8">

<meta

name="viewport"

content="width=device-width, initial-scale=1.0"

>

<title>My Website</title>

<link

rel="stylesheet"

href="style.css"

>

`</head>`

Exam Definition

The HTML `<head>` element contains metadata and resources that describe or support the HTML document, such as its title, character encoding, viewport settings, CSS files, icons, and scripts.

HTML `<head>` Part — Complete Notes

1. What is the `<head>` Element?

The `<head>` element contains information and resources about an HTML document that are generally not displayed as the main page content.

It is placed between `<html>` and `<body>`.

Basic Structure

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <!-- Metadata and resources -->
```

```
</head>
```

```
<body>
```

```
  <!-- Visible webpage content -->
```

```
</body>
```

</html>

2. Basic Syntax

<head>

<meta charset="UTF-8">

<meta

name="viewport"

content="width=device-width, initial-scale=1.0"

>

<title>My Website</title>

</head>

3. Important Elements Used Inside

<head>

Element	Purpose
---------	---------

<code><title></code>	Sets the page title
<code><meta></code>	Provides metadata
<code><link></code>	Connects external resources
<code><style></code>	Contains internal CSS
<code><script></code>	Includes or loads JavaScript
<code><base></code>	Sets a base URL for relative URLs

4. `<title>`

The `<title>` element defines the title of the HTML document.

It is commonly shown in:

- Browser tab
- Bookmarks
- Search engine results

Syntax

```
<title>Page Title</title>
```

Example

```
<head>
```

```
  <title>My Portfolio</title>
```

</head>

5. <meta>

<meta> provides metadata about the document.

Metadata is information about the webpage that isn't normally displayed as page content.

A. Character Encoding

Use:

<meta charset="UTF-8">

UTF-8 supports a very large range of characters and is the standard choice for most modern HTML documents.

Example

<head>

<meta charset="UTF-8">

<title>My Website</title>

</head>

6. Viewport Meta Tag

This is particularly important for responsive websites.

Syntax

```
<meta  
  name="viewport"  
  content="width=device-width, initial-scale=1.0"  
>
```

Meaning

width=device-width



Use the device's viewport width

initial-scale=1.0



Initial zoom level = 1

Without an appropriate viewport configuration, mobile browsers may render pages in a way that makes responsive layouts behave unexpectedly.

7. Meta Description

A description can provide a summary of the page.

Example

```
<meta  
  name="description"
```

```
content="Learn HTML, CSS and JavaScript."
```

```
>
```

This is metadata that can be used by search engines when generating search-result snippets, although search engines decide what snippet to display.

8. Meta Keywords

You may see old HTML tutorials using:

```
<meta
```

```
name="keywords"
```

```
content="HTML, CSS, JavaScript"
```

```
>
```

However, major search engines generally do not use the **keywords** meta tag for ranking. It is not something you need for modern HTML development.

9. **<link>**

The **<link>** element connects the HTML document to an external resource.

The most common use is connecting an external CSS file.

Syntax

```
<link rel="stylesheet" href="style.css">
```

Example

```
<head>
```

```
<title>My Website</title>
```

```
<link
```

```
  rel="stylesheet"
```

```
  href="style.css"
```

```
>
```

```
</head>
```

10. Favicon

A favicon is the small icon associated with a webpage, often shown in the browser tab.

Example

```
<link
```

```
  rel="icon"
```

```
  href="favicon.png"
```

```
>
```

You can also specify a MIME type when useful:

```
<link
```

```
  rel="icon"
```

```
  type="image/png"
```

```
  href="favicon.png"
```

```
>
```

11. <style>

The <style> element contains internal CSS.

Example

```
<head>
```

```
  <style>
```

```
    body {  
      font-family: Arial, sans-serif;  
    }
```

```
    h1 {  
      color: blue;  
    }
```

```
  </style>
```

```
</head>
```

This is different from the style attribute:

Inline CSS

```
<h1 style="color: blue;">
```

```
  Hello
```

</h1>

Internal CSS

<style>

```
h1 {  
    color: blue;  
}
```

</style>

External CSS

<link rel="stylesheet" href="style.css">

For larger projects, external CSS is generally easier to maintain.

12. <script>

The <script> element is used to include or embed JavaScript.

Example:

```
<script src="script.js"></script>
```

You can also put JavaScript directly inside it:

<script>

```
    console.log("Hello World");
```

`</script>`

For scripts that can be loaded without blocking HTML parsing, `defer` is commonly useful when loading an external script:

`<script src="script.js" defer></script>`

13. `<base>`

The `<base>` element specifies the base URL used for relative URLs in the document.

Example

`<base href="https://example.com/">`

Then:

`<a href="about.html">About</a>`

can resolve relative to that base URL.

`<base>` is less commonly needed in basic HTML projects, but it is part of the `<head>` knowledge.

14. Complete `<head>` Example

`<!DOCTYPE html>`

`<html lang="en">`

`<head>`

```
<!-- Character Encoding -->

<meta charset="UTF-8">

<!-- Responsive Design -->

<meta

    name="viewport"

    content="width=device-width, initial-scale=1.0"

>

<!-- Page Title -->

<title>Raj's Portfolio</title>

<!-- Page Description -->

<meta

    name="description"

    content="Personal portfolio website"

>

<!-- Favicon -->

<link

    rel="icon"

    type="image/png"

    href="favicon.png"

>

<!-- External CSS -->

<link

    rel="stylesheet"

    href="style.css"

>

<!-- Internal CSS -->
```

```
<style>
  body {
    margin: 0;
  }
</style>

<!-- JavaScript -->

<script
  src="script.js"
  defer
></script>
</head>
<body>
  <h1>My Portfolio</h1>
</body>
</html>
```

15. <head> vs <body>

This is important for exams.

<head>

Contains document metadata and resources

<body>

Contains page content

Usually not directly visible

Visible webpage content

Contains `<title>`

Contains headings, paragraphs, images, etc.

Contains `<meta>`

Contains `<p>`, `<h1>`, `<img>`, etc.

Contains `<link>`

Contains links such as `<a>`

Can contain `<style>`

Can contain visible content

Can contain `<script>`

Can also contain scripts

Simple way to remember

`<head>`

Information ABOUT the webpage

`</head>`

`<body>`

Content OF the webpage

`</body>`

16. Complete HTML Document Structure

`<!DOCTYPE html>`

`<html lang="en">`

<head>

<meta charset="UTF-8">

<meta

name="viewport"

content="width=device-width, initial-scale=1.0"

>

<title>My Website</title>

<link

rel="stylesheet"

href="style.css"

>

</head>

<body>

<header>

<h1>My Website</h1>

</header>

<main>

<section>

<h2>Welcome</h2>

<p>

This is my webpage.

</p>

</section>

</main>

<footer>

<p>© 2026 My Website</p>

</footer>

</body>

</html>

Quick Revision

<head>



Document metadata + resources

<title> → Page title

<meta> → Metadata

<link> → External resources

<style> → Internal CSS

<script> → JavaScript

<base> → Base URL

Most Important Syntax

<head>

<meta charset="UTF-8">

<meta

name="viewport"

content="width=device-width, initial-scale=1.0"

>

<title>My Website</title>

<link

rel="stylesheet"

href="style.css"

>

</head>

Exam Definition

The HTML **<head>** element contains metadata and resources that describe or support the HTML document, such as its title, character encoding, viewport settings, CSS files, icons, and scripts.

HTML Style Guide — Complete Notes

1. What is an HTML Style Guide?

An HTML Style Guide is a set of rules and best practices for writing clean, consistent, readable, and maintainable HTML code.

A style guide helps developers follow the same coding conventions across a project.

Important: HTML generally does not require a single formatting style to work. A style guide is mainly about code quality, consistency, and readability.

2. Use **<!DOCTYPE html>**

Always start a modern HTML document with the HTML5 doctype.

Example

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <title>My Website</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Hello World</h1>
```

```
</body>
```

```
</html>
```

3. Use the **lang** Attribute

Specify the primary language of the document.

Recommended

```
<html lang="en">
```

For Hindi:

```
<html lang="hi">
```

This helps browsers and assistive technologies understand the document language.

4. Use Proper Indentation

Indent nested elements to make the structure easy to understand.

Good

```
<body>
```

```
  <header>
```

```
    <h1>My Website</h1>
```

```
    <nav>
```

```
      <a href="#home">Home</a>
```

```
      <a href="#about">About</a>
```

```
    </nav>
```

```
  </header>
```

```
</body>
```

Difficult to Read

```
<body><header><h1>My Website</h1><nav><a  
href="#home">Home</a></nav></header></body>
```

Both may work, but the first is much easier to maintain.

5. Use Lowercase Element Names

HTML is generally case-insensitive for element names, but lowercase is the common modern convention.

Recommended

```
<h1>Welcome</h1>
```

```
<p>Hello World</p>
```

Avoid

```
<H1>Welcome</H1>
```

```
<P>Hello World</P>
```

6. Use Lowercase Attribute Names

Use lowercase for attributes.

Recommended

```

```

Instead of:

```
<img SRC="photo.jpg" ALT="Student Photo">
```

7. Use Double Quotes for Attribute Values

Double quotes are the standard convention.

Recommended

```
<a href="about.html">About</a>
```

```

```

8. Always Use Meaningful **alt** Text

Images should generally have an **alt** attribute.

Good

```

```

The **alt** text should describe the image's purpose/content.

For a purely decorative image, an empty alternative text can be appropriate:

```

```

9. Use Semantic HTML

Prefer elements that describe the meaning of the content.

Recommended

```
<header>
```

```
  <nav>
```

```
    ...
```

```
  </nav>
```

```
</header>
```

```
<main>
```

```
  <section>
```

```
    <article>
```

```
      ...
```

```
    </article>
```

```
  </section>
```

```
</main>
```

```
<footer>
```

```
  ...
```

```
</footer>
```

Instead of using `<div>` for every part:

```
<div>
```

```
  <div>
```

```
    ...
```

`</div>`

`</div>`

Semantic HTML improves structure and accessibility.

10. Use Proper Heading Hierarchy

Use headings according to the document structure.

`<h1>My Website</h1>`

`<h2>About Me</h2>`

`<h3>Education</h3>`

`<h3>Skills</h3>`

`<h2>Projects</h2>`

`<h3>Project 1</h3>`

Don't select a heading simply because its default font size looks good. Use CSS when you need to change visual size.

11. Use Meaningful Names for **id** and **class**

Good

```
<section id="about">
```

```
<div class="student-card">
```

Less Meaningful

```
<section id="x1">
```

```
<div class="box1">
```

Names should communicate the element's purpose.

12. Keep HTML and CSS Separate

For small experiments, inline styles are okay:

```
<p style="color: blue;">
```

Hello

```
</p>
```

For larger projects, prefer external CSS:

```
<link rel="stylesheet" href="style.css">
```

Then:

```
p {
```

```
  color: blue;
```

```
}
```

This keeps the HTML focused on structure and the CSS focused on presentation.

13. Use Comments Properly

HTML comments are written as:

```
<!-- This is a comment -->
```

Example

```
<header>
```

```
    <!-- Website navigation -->
```

```
    <nav>
```

```
        <a href="#home">Home</a>
```

```
        <a href="#about">About</a>
```

```
    </nav>
```

```
</header>
```

Comments should explain something useful rather than describe every obvious line.

14. Use meta charset

A modern HTML document should normally specify UTF-8.

```
<meta charset="UTF-8">
```

This helps the browser interpret characters correctly.

15. Use the Viewport Meta Tag

For responsive webpages:

```
<meta  
  name="viewport"  
  content="width=device-width, initial-scale=1.0"  
>
```

This is especially important when designing for mobile devices.

16. Close Elements Properly

Elements that have closing tags should be properly closed.

Good

```
<p>This is a paragraph.</p>
```

```
<div>
```

Content

```
</div>
```

Some HTML elements are void elements and do not have closing tags.

Examples:

```

```

`<br>`

`<hr>`

`<input type="text">`

17. Use Valid HTML Nesting

Elements should be nested correctly.

Good

`<p>`

 This is `<strong>important</strong>` text.

`</p>`

Incorrect nesting

`<p>`

 This is `<strong>important text.`

`</p>`

`</strong>`

Think of HTML elements as nested containers.

18. Use Spaces Around Attributes

Recommended

```

```

Avoid unnecessary or inconsistent spacing:

```

```

19. Keep Lines Readable

If an element has many attributes, you can place them on separate lines.

Example

```

```

This makes long elements easier to read.

20. Use Accessibility-Friendly HTML

Good HTML should be usable by people using assistive technologies.

Examples:

Form label

```
<label for="email">
```

Email:

</label>

<input

type="email"

id="email"

name="email"

>

Image alternative text

Semantic button

Use:

<button type="button">

Submit

</button>

rather than using a generic `<div>` to imitate a button.

21. Use External JavaScript Properly

Instead of writing a large amount of JavaScript directly in HTML:

```
<script src="script.js" defer></script>
```

This keeps your project organized.

A common project structure is:

project/

|

|— index.html

|— style.css

|— script.js

22. Complete HTML Style Guide Example

Here is a clean, well-organized HTML document:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta
```

```
    name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
```

```
>
```

```
  <meta
```

```
    name="description"
```

```
    content="A simple student portfolio website"
```

```
>
```

```
<title>Student Portfolio</title>
```

```
<link
```

```
    rel="stylesheet"
```

```
    href="style.css"
```

```
>
```

```
<script
```

```
    src="script.js"
```

```
    defer
```

```
></script>
```

```
</head>
```

```
<body>
```

```
    <header>
```

```
        <h1>Student Portfolio</h1>
```

```
        <nav aria-label="Main navigation">
```

```
            <a href="#about">About</a>
```

```
            <a href="#skills">Skills</a>
```

```
            <a href="#projects">Projects</a>
```

```
            <a href="#contact">Contact</a>
```

```
        </nav>
```

```
    </header>
```

```
    <main>
```

```
        <section id="about">
```

```
            <h2>About Me</h2>
```

```
            <p>
```

```
                I am a student learning web development.
```

```
            </p>
```

</section>

<section id="skills">

<h2>Skills</h2>

HTML

CSS

JavaScript

</section>

<section id="projects">

<h2>Projects</h2>

<article>

<h3>Student Management System</h3>

<p>

A web project for managing student information.

</p>

</article>

</section>

<section id="contact">

<h2>Contact</h2>

<form>

<label for="name">

Name:

</label>

<input

type="text"

```
        id="name"
        name="name"
    >
    <button type="submit">
        Submit
    </button>
</form>
</section>
</main>
<footer>
    <p>
        © 2026 Student Portfolio
    </p>
</footer>
</body>
</html>
```



localhost:54188/storage/emulatec



Student Portfolio

[About](#) [Skills](#) [Projects](#) [Contact](#)

About Me

I am a student learning web development.

Skills

- HTML
- CSS
- JavaScript

Projects

Student Management System

A web project for managing student information.

Contact

Name:

© 2026 Student Portfolio

23. HTML Style Guide — Quick Revision

Remember these rules:

1. Use `<!DOCTYPE html>`
2. Use `<html lang="en">`

3. Use lowercase HTML elements
4. Use lowercase attributes
5. Use double quotes for attribute values
6. Indent nested elements
7. Use semantic HTML
8. Use meaningful id/class names
9. Use proper heading hierarchy
10. Add useful alt text to images
11. Use labels with form controls
12. Keep HTML, CSS and JS organized
13. Use comments when they add useful context
14. Maintain proper nesting
15. Use viewport meta for responsive pages
16. Keep code readable and consistent

Exam Definition

An HTML Style Guide is a set of coding conventions and best practices used to make HTML code consistent, readable, accessible, maintainable, and easier to understand.

Images in HTML — Complete Notes

1. What are Images in HTML?

Images can be added to a webpage using the `<img>` element.

Images make webpages more informative and visually attractive. They can be used for:

- Photos
- Logos
- Icons

- Diagrams
 - Product images
 - Banners
 - Illustrations
-

2. `<img>` Element

The `<img>` element is used to embed an image in an HTML document.

Syntax

```

```

`<img>` is a void element, so it does not have a closing `</img>` tag.

3. `src` Attribute

`src` stands for source.

It specifies the location/path of the image.

Example

```

```

Here:

```
src="photo.jpg"
```

↓

Image file location

4. alt Attribute

alt stands for alternative text.

It describes the image when the image cannot be displayed and is also important for accessibility.

Example

```

```

Good alt

```

```

Avoid meaningless alt text

```

```

If an image is purely decorative, an empty **alt** is appropriate:

```

```

5. Image Width and Height

You can specify image dimensions using the **width** and **height** attributes.

Syntax

```

```

The values are CSS pixels when specified as HTML numeric dimensions.

Example

```

```

Providing correct intrinsic dimensions can also help the browser reserve space for the image while the page loads.

6. Image from Same Folder

Suppose your project is:

```
project/
|
|— index.html
|— photo.jpg
```

Use:

``

7. Image from a Folder

Suppose:

project/

|

|— index.html

|— images/

 |— student.jpg

Use:

`<img`

`src="images/student.jpg"`

`alt="Student Photo"`

`>`

8. Image from Parent Folder

Suppose:

project/

|

|— index.html

|— pages/

└─ about.html

If **about.html** needs an image in the project root:

```

```

../ means go up one directory.

9. Image Using an Absolute URL

You can also reference an image using a complete URL:

```

```

However, using images hosted on other websites can be unreliable because the URL may change or the server may block external embedding. For your own projects, storing images with your project is often preferable.

10. Common Image Formats

Format

Common Use

.jpg / .jpeg

Photographs

.png

Graphics and transparency

.gif

Simple animations

.svg

Logos, icons, vector graphics

.webp

Modern web images

.avif

Modern, highly compressed images

Example:

```

```

11. Image as a Link

An image can be placed inside an **<a>** element to make it clickable.

Example

```
<a href="https://example.com">
```

```
  
```


The user can click the image to follow the link.

12. Styling Images with CSS

CSS can control the appearance of images.

Example

```

```

This applies:

- `width: 300px`
 - `border-radius: 10px`
-

13. Responsive Images

For an image to adapt to the size of its container:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Example

```

```

This helps prevent the image from overflowing its container on smaller screens.

14. Image Alignment

Modern HTML uses CSS for alignment.

Center an Image

```
img {  
  display: block;  
  margin: 0 auto;  
}
```

HTML:

```

```

15. <figure> and <figcaption>

When an image is self-contained content with a caption, use <figure> and <figcaption>.

Example

<figure>

```

```

```
<figcaption>  
  HTML Logo  
</figcaption>
```

</figure>

Meaning

<figure>

↓

Image/content

<figcaption>

↓

16. Image with Caption — Practical Example

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Images in HTML</title>

</head>

<body>

    <h1>My Image</h1>

    <figure>

        
```

```
<figcaption>
```

```
    A student learning web development.
```

```
</figcaption>
```

```
</figure>
```

```
</body>
```

```
</html>
```

17. **<picture>** Element

The **<picture>** element allows you to provide different image sources depending on conditions such as screen size or supported formats.

Example

```
<picture>
```

```
    <source
```

```
        media="(max-width: 600px)"
```

```
        srcset="mobile.jpg"
```

```
    >
```

```
    

</picture>

Here:

- Smaller screens can use `mobile.jpg`.
  - Other situations use `desktop.jpg`.
- 

## 18. `srcset` for Responsive Images

`srcset` allows the browser to choose from multiple image versions.

### Example

```

```

This can help browsers select an appropriate image resource for the display size and resolution.

---

## 19. Lazy Loading

For images that are not immediately needed, you can use:

```

```

**loading="lazy"** tells the browser that the image can be loaded later when it approaches the viewport.

For important above-the-fold images, lazy loading is usually not desirable.

---

## 20. Image Accessibility

Always consider accessibility when adding images.

### Informative image

```

```

### Decorative image

```

```

The **alt** text should communicate the purpose or meaningful information of the image, not merely say "image."

---

## 21. Complete Practical Program

This program demonstrates several concepts:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
 <meta charset="UTF-8">
```

```
 <meta
```

```
 name="viewport"
```

```
 content="width=device-width, initial-scale=1.0"
```

```
 >
```

```
<title>Images in HTML</title>
```

```
<style>
```

```
 body {
```

```
 font-family: Arial, sans-serif;
 text-align: center;
}
```

```
figure {
 max-width: 500px;
 margin: 30px auto;
}
```

```
img {
 max-width: 100%;
 height: auto;
 border-radius: 12px;
}
```

```
figcaption {
 margin-top: 10px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Images in HTML</h1>
```

```
<figure>
```

```

```

```
<figcaption>
```

```
 Student learning web development
```

```
</figcaption>
```

```
</figure>
```

```
</body>
```

```
</html>
```

---

## 22. Important Image Attributes

Attribute	Purpose	Example
-----------	---------	---------

<b>src</b>	Image location	<b>src="photo.jpg"</b>
<b>alt</b>	Alternative text	<b>alt="Student"</b>
<b>width</b>	Image width	<b>width="300"</b>
<b>height</b>	Image height	<b>height="200"</b>
<b>srcset</b>	Multiple image sources	<b>srcset="small.jpg 400w"</b>
<b>sizes</b>	Helps choose responsive source	<b>sizes="100vw"</b>
<b>loading</b>	Loading behavior	<b>loading="lazy"</b>

---

## 23. Quick Revision

### Basic Image

```

```

### Image with Size

```

```

## Responsive Image

```

```

## Image with Caption

```
<figure>

 <figcaption>
 My Photo
 </figcaption>

</figure>
```

## Exam Definition

The HTML `<img>` element is used to embed images in a webpage. The `src` attribute specifies the image source, while the `alt` attribute provides alternative text describing the image.

# Video in HTML

## Definition

The HTML5 `<video>` element is a native multimedia container used to embed, stream, and play video files directly within a web page without requiring external third-party plugins (such as Adobe Flash Player or Silverlight).

## Syntax

### Basic Syntax

#### HTML

```
video src "media/sample.mp4" width "640" height "360" controls
```

```
video
```

### Multi-Format Fallback Syntax (Standard Production Practice)

#### HTML

```
video width "640" height "360" controls poster "images/thumbnail.jpg"
```

```
source src "media/sample.mp4" type "video/mp4"
```

```
source src "media/sample.webm" type "video/webm"
```

```
source src "media/sample.ogv" type "video/ogg"
```

# Explanation

## 1. Key Attributes of the `<video>` Element

Attribute	Type	Description
<code>src</code>	URL	Specifies the path to the video file (optional if <code>&lt;source&gt;</code> tags are used).
<code>controls</code>	Boolean	Displays built-in browser playback controls (play, pause, volume, full-screen, seek bar).
<code>autoplay</code>	Boolean	Starts video playback automatically once loaded.
<code>muted</code>	Boolean	Mutes audio output by default. Note: Modern browsers require <code>muted</code> for <code>autoplay</code> to function.
<code>loop</code>	Boolean	Automatically restarts the video from the beginning when it finishes playing.
<code>poster</code>	URL	Displays an image placeholder before the user clicks play or while the video is downloading.
<code>preload</code>	String	Suggests how the browser should load the file: <code>auto</code> , <code>metadata</code> , or <code>none</code> .
<code>width / height</code>	Pixels	Sets the display dimensions of the video player in pixels.

<code>playsinline</code>	Boolean	Plays inline on mobile devices rather than opening a fullscreen player automatically.
--------------------------	---------	---------------------------------------------------------------------------------------

## 2. Supported Video Formats and MIME Types

Different browsers support different video codecs. Providing multiple sources inside the `<video>` element ensures cross-browser compatibility.

Format	Codec	MIME Type	Browser Support
MP4	H.264 + AAC	<code>video/mp4</code>	Universal (Chrome, Edge, Firefox, Safari, Opera)
WebM	VP8/VP9 + Vorbis/Opus	<code>video/webm</code>	Chrome, Firefox, Edge, Opera
OGG	Theora + Vorbis	<code>video/ogg</code>	Firefox, Chrome, Opera

## 3. The `<source>` and `<track>` Elements

- `<source>` Tag: An empty (void) element placed inside `<video>` that allows specifying alternative media files. The browser checks them top-to-bottom and plays the first format it supports.
- `<track>` Tag: Used to attach text tracks such as subtitles, closed captions (`kind="captions"`), or chapter markers (`kind="chapters"`) in WebVTT format (`.vtt`).
- Fallback Text: Text placed before `</video>` that renders only if the browser does not support HTML5 video.
- 

## Examples

### Example 1: Background Video (Autoplay, Loop, Muted)

#### HTML

```
<!-- Background video loop with muted audio so autoplay succeeds -->
```

```
video width "100%" height "auto" autoplay loop muted playsinline
```

```
source src "background.mp4" type "video/mp4"
```

```
source src "background.webm" type "video/webm"
```

video

## Example 2: Video with Subtitles / Captions

### HTML

```
video width "640" height "360" controls poster "thumbnail.jpg"

 source src "lecture.mp4" type "video/mp4"

 track src "captions_en.vtt" kind "subtitles" srclang "en" label "English"
```

video

## Practical Program

Save this code into an `index.html` file to test standard video player controls, custom poster display, and responsive video styling:

### HTML

```
<!DOCTYPE html>

html lang "en"

head
 meta charset "UTF-8"
 meta name "viewport" content "width=device-width, initial-scale=1.0"
 title
 style
 body

 #f8f9fa
 20px
 #333
 .container
 720px
```

```
0
#ffffff
20px
8px
0 2px 8px rgba 0 0 0 0.1

/* Responsive Video Container */
.video-wrapper
 100%
 15px
 6px
 #000
video
 100%

.video-details
 12px
 0.95rem
 #555

style
head
```

body

div class "container"

h1

h1

p

p

<!-- HTML5 Video Element -->

div class "video-wrapper"

video

controls

poster "https://images.unsplash.com/photo-1518770660439-4636190af475?w=720&q=80"

preload "metadata"

<!-- Multi-format Sources -->

source

src "https://commondatastorage.googleapis.com/gtv-videos-bucket/sample/BigBuckBunny.mp4"

type "video/mp4"

source

src "https://commondatastorage.googleapis.com/gtv-videos-bucket/sample/BigBuckBunny.webm"

" type "video/webm"

<!-- Fallback content for older/unsupported browsers -->

p

p

video

div

div class "video-details"

```
code p strong code strong code code
 code code code code code p code

div

div

body

html
```

## Important Points

- **Autoplay Restrictions:** Modern web browsers block unmuted audio playback on load. If you use `autoplay`, you must also include `muted` (`<video autoplay muted>`), otherwise the browser will refuse to autoplay.
- 
- **Dimensions vs Ratio:** Always set `width` and `height` (or CSS `max-width: 100%`) so the browser reserves layout space before the video data streams, avoiding sudden layout shifts (CLS).
- 
- **Obsolete Features:** The old HTML4 `<embed>` or `<object>` tags for plugin-based media players are obsolete for video. Always use the native HTML5 `<video>` tag.
- 

## Common Mistakes

1. **Forgetting the `controls` Attribute:** If `controls` is omitted and `autoplay` is not enabled, the video will appear as a frozen, unplayable image with no user interface.
- 2.
3. **Assuming MP4 works everywhere without specifying MIME types:** Omitting `type="video/mp4"` inside `<source>` forces the browser to download file headers first to determine if it can play the file, causing delayed playback.
- 4.
5. **Using `autoplay` without `muted`:** Expecting videos with sound to play automatically on page load. Browser security policies will block the video from playing.
- 6.

## Exam Definition

**HTML5 `<video>` Tag:** A standard HTML5 container element used to embed and playback video content directly inside a web document without external browser plugins. It provides native controls via the `controls` attribute and supports multi-format source declarations using nested `<source>` tags for cross-platform compatibility.

## Quick Revision

- Main Element: `<video> ... </video>`
- 
- Source Delivery: Use `<source src="file.ext" type="mime/type">` inside `<video>` for multi-codec fallback.
- 
- Core Attributes: `controls` (UI controls), `poster` (preview image), `muted` (mute audio), `autoplay` (auto start), `loop` (infinite replay).
- 
- Subtitles/Captions: Added via the self-closing `<track kind="subtitles" src="subs.vtt" srclang="en">` element.

# Video in HTML

## Definition

The HTML5 `<video>` element is a native multimedia container used to embed, stream, and play video files directly within a web page without requiring external third-party plugins (such as Adobe Flash Player or Silverlight).

## Syntax

### Basic Syntax

#### HTML

```
video src "media/sample.mp4" width "640" height "360" controls
```

```
video
```

### Multi-Format Fallback Syntax (Standard Production Practice)

#### HTML

```
video width "640" height "360" controls poster "images/thumbnail.jpg"
```

```
source src "media/sample.mp4" type "video/mp4"
```

```
source src "media/sample.webm" type "video/webm"
```

```
source src "media/sample.ogv" type "video/ogg"
```

# Explanation

## 1. Key Attributes of the `<video>` Element

Attribute	Type	Description
<code>src</code>	URL	Specifies the path to the video file (optional if <code>&lt;source&gt;</code> tags are used).
<code>controls</code>	Boolean	Displays built-in browser playback controls (play, pause, volume, full-screen, seek bar).
<code>autoplay</code>	Boolean	Starts video playback automatically once loaded.
<code>muted</code>	Boolean	Mutes audio output by default. Note: Modern browsers require <code>muted</code> for <code>autoplay</code> to function.
<code>loop</code>	Boolean	Automatically restarts the video from the beginning when it finishes playing.
<code>poster</code>	URL	Displays an image placeholder before the user clicks play or while the video is downloading.
<code>preload</code>	String	Suggests how the browser should load the file: <code>auto</code> , <code>metadata</code> , or <code>none</code> .
<code>width / height</code>	Pixels	Sets the display dimensions of the video player in pixels.

<code>playsinline</code>	Boolean	Plays inline on mobile devices rather than opening a fullscreen player automatically.
--------------------------	---------	---------------------------------------------------------------------------------------

## 2. Supported Video Formats and MIME Types

Different browsers support different video codecs. Providing multiple sources inside the `<video>` element ensures cross-browser compatibility.

Format	Codec	MIME Type	Browser Support
MP4	H.264 + AAC	<code>video/mp4</code>	Universal (Chrome, Edge, Firefox, Safari, Opera)
WebM	VP8/VP9 + Vorbis/Opus	<code>video/webm</code>	Chrome, Firefox, Edge, Opera
OGG	Theora + Vorbis	<code>video/ogg</code>	Firefox, Chrome, Opera

## 3. The `<source>` and `<track>` Elements

- `<source>` Tag: An empty (void) element placed inside `<video>` that allows specifying alternative media files. The browser checks them top-to-bottom and plays the first format it supports.
- `<track>` Tag: Used to attach text tracks such as subtitles, closed captions (`kind="captions"`), or chapter markers (`kind="chapters"`) in WebVTT format (`.vtt`).
- Fallback Text: Text placed before `</video>` that renders only if the browser does not support HTML5 video.
- 

## Examples

### Example 1: Background Video (Autoplay, Loop, Muted)

#### HTML

```
<!-- Background video loop with muted audio so autoplay succeeds -->
```

```
video width "100%" height "auto" autoplay loop muted playsinline
```

```
source src "background.mp4" type "video/mp4"
```

```
source src "background.webm" type "video/webm"
```

video

## Example 2: Video with Subtitles / Captions

### HTML

```
video width "640" height "360" controls poster "thumbnail.jpg"

 source src "lecture.mp4" type "video/mp4"

 track src "captions_en.vtt" kind "subtitles" srclang "en" label "English"
```

video

## Practical Program

Save this code into an `index.html` file to test standard video player controls, custom poster display, and responsive video styling:

### HTML

```
<!DOCTYPE html>

html lang "en"

head

 meta charset "UTF-8"

 meta name "viewport" content "width=device-width, initial-scale=1.0"

 title title

 style

 body

 #f8f9fa

 20px

 #333

 .container

 720px
```

```
0
#ffffff
20px
8px
0 2px 8px rgba(0, 0, 0, 0.1)
```

```
/* Responsive Video Container */
```

```
.video-wrapper
```

```
100%
15px
6px
#000
```

```
video
```

```
100%
```

```
.video-details
```

```
12px
0.95rem
#555
```

```
style
```

```
head
```

body

div class "container"

h1

h1

p

p

<!-- HTML5 Video Element -->

div class "video-wrapper"

video

controls

poster "https://images.unsplash.com/photo-1518770660439-4636190af475?w=720&q=80"

preload "metadata"

<!-- Multi-format Sources -->

source

src "https://commondatastorage.googleapis.com/gtv-videos-bucket/sample/BigBuckBunny.mp4"

type "video/mp4"

source

src "https://commondatastorage.googleapis.com/gtv-videos-bucket/sample/BigBuckBunny.webm"

type "video/webm"

<!-- Fallback content for older/unsupported browsers -->

p

p

video

div

div class "video-details"

```
code p strong code strong code code
 code code code code code p code

div

div

body

html
```

## Important Points

- **Autoplay Restrictions:** Modern web browsers block unmuted audio playback on load. If you use `autoplay`, you must also include `muted` (`<video autoplay muted>`), otherwise the browser will refuse to autoplay.
- 
- **Dimensions vs Ratio:** Always set `width` and `height` (or CSS `max-width: 100%`) so the browser reserves layout space before the video data streams, avoiding sudden layout shifts (CLS).
- 
- **Obsolete Features:** The old HTML4 `<embed>` or `<object>` tags for plugin-based media players are obsolete for video. Always use the native HTML5 `<video>` tag.
- 

## Common Mistakes

1. **Forgetting the `controls` Attribute:** If `controls` is omitted and `autoplay` is not enabled, the video will appear as a frozen, unplayable image with no user interface.
- 2.
3. **Assuming MP4 works everywhere without specifying MIME types:** Omitting `type="video/mp4"` inside `<source>` forces the browser to download file headers first to determine if it can play the file, causing delayed playback.
- 4.
5. **Using `autoplay` without `muted`:** Expecting videos with sound to play automatically on page load. Browser security policies will block the video from playing.
- 6.

## Exam Definition

**HTML5 `<video>` Tag:** A standard HTML5 container element used to embed and playback video content directly inside a web document without external browser plugins. It provides native controls via the `controls` attribute and supports multi-format source declarations using nested `<source>` tags for cross-platform compatibility.

## Quick Revision

- Main Element: `<video> ... </video>`
- 
- Source Delivery: Use `<source src="file.ext" type="mime/type">` inside `<video>` for multi-codec fallback.
- 
- Core Attributes: `controls` (UI controls), `poster` (preview image), `muted` (mute audio), `autoplay` (auto start), `loop` (infinite replay).
- 
- Subtitles/Captions: Added via the self-closing `<track kind="subtitles" src="subs.vtt" srclang="en">` element.

## Audio in HTML

### Definition

The HTML5 `<audio>` element is a native multimedia container used to embed, stream, and play sound files, music, podcasts, or voice recordings directly within a web page without requiring third-party plugins like Flash or Silverlight.

### Syntax

#### Basic Syntax

HTML

```
audio src "media/podcast.mp3" controls
```

```
audio
```

#### Multi-Format Fallback Syntax (Recommended Standard)

HTML

```
audio controls preload "metadata"
```

```
source src "media/podcast.mp3" type "audio/mpeg"
```

```
source src "media/podcast.ogg" type "audio/ogg"
```

```
source src "media/podcast.wav" type "audio/wav"
```

```
audio
```

## Explanation

### 1. Key Attributes of the `<audio>` Element

Attribute	Type	Description
<code>src</code>	URL	Path to the sound file (optional if <code>&lt;source&gt;</code> tags are nested inside).
<code>controls</code>	Boolean	Renders the browser's built-in playback interface (Play, Pause, Volume, Seek bar, Time display).
<code>autoplay</code>	Boolean	Starts playing the audio automatically as soon as it is ready.
<code>muted</code>	Boolean	Sets the audio output to silent by default.
<code>loop</code>	Boolean	Replays the audio continuously from the beginning once it ends.
<code>preload</code>	String	Gives a hint to the browser on how to load the file: <code>auto</code> , <code>metadata</code> , or <code>none</code> .

## 2. Preload Values Breakdown

- `preload="auto"`: Instructs the browser to download the entire audio file when the page loads (best if audio is the primary content).
- `preload="metadata"`: Downloads only file metadata (duration, track title, audio size) to save user bandwidth.
- `preload="none"`: Tells the browser not to buffer the audio file until the user presses play (saves maximum bandwidth).
- 

## 3. Supported Audio Formats and MIME Types

To ensure every browser plays the audio regardless of platform, use the `<source>` tag with proper MIME types.

Format	File Extension	MIME Type	Browser Support
MP3	.mp3	audio/mpeg	Universal (Chrome, Firefox, Safari, Edge, Opera)
OGG	.ogg, .oga	audio/ogg	Chrome, Firefox, Opera, Edge
WAV	.wav	audio/wav	Universal (high quality, but large file size)
AAC / M4A	.aac, .m4a	audio/mp4 / audio/aac	Safari, Chrome, Edge

## Examples

### Example 1: Basic Audio Player with Controls and Preload

#### HTML

```
<!-- Simple audio player with metadata loading -->

audio controls preload "metadata"

 source src "audio/lecture-01.mp3" type "audio/mpeg"

p
p

audio
```

### Example 2: Autoplay and Continuous Loop (Muted)

#### HTML

```
<!-- Muted background audio loop -->

audio autoplay loop muted

 source src "audio/ambient-sound.ogg" type "audio/ogg"

 source src "audio/ambient-sound.mp3" type "audio/mpeg"
```

audio

## Practical Program

Save and open the following complete HTML file to test an interactive audio player with controls, file details, and styling:

### HTML

```
<!DOCTYPE html>

html lang "en"

head
 meta charset "UTF-8"
 meta name "viewport" content "width=device-width, initial-scale=1.0"
 title
 style
 body

 #f0f2f5
 30px
 #222

 .audio-card
 500px
 0
 #ffffff
 24px
 10px
 0 4px 12px rgba 0 0 0 0.1

 .audio-card h2
```

0

#0d6efd

1.4rem

.audio-card p

0.95rem

#555

1.5

audio

100%

15px

.info-box

15px

10px 14px

#e9ecef

4px

#0d6efd

0.85rem

style

head

body

div class "audio-card"

h2

h2

```
p
code < > code
p
```

```
<!-- HTML5 Audio Component with Multiple Formats -->
```

```
audio controls preload "metadata"
```

```
<!-- Source 1: Standard MP3 Format -->
```

```
source src "https://www.w3schools.com/html/horse.mp3" type "audio/mpeg"
```

```
<!-- Source 2: OGG Fallback Format -->
```

```
source src "https://www.w3schools.com/html/horse.ogg" type "audio/ogg"
```

```
<!-- Fallback text for older non-HTML5 browsers -->
```

```
p
p
audio
```

```
div class "info-box"
```

```
strong
code
strong
code
br
strong
code
code
strong
code
code
div
div
```

```
body
```

```
html
```

## Important Points

- **Invisible by Default:** An `<audio>` element without the `controls` attribute will not be rendered visually on the web page at all (unless controlled programmatically via JavaScript).
-

- **Browser Autoplay Policy:** Browsers block unmuted audio from autoplaying automatically to protect user experience. To test `autoplay`, you must pair it with `muted` (`<audio autoplay muted>`) or trigger it through a direct user interaction (like a button click).
- 
- **Obsolete Elements:** Never use obsolete tags like `<bgsound>`, `<embed>`, or `<applet>` for web audio. The `<audio>` tag is the official W3C and WHATWG standard.
- 

## Common Mistakes

1. **Omitting the `controls` Attribute:** Writing `<audio src="music.mp3"></audio>` without `controls`. The audio exists in the DOM but the user has no interface to play, pause, or adjust volume.
- 2.
3. **Incorrect MIME Type Declaration:** Writing `type="audio/mp3"` instead of the standard `type="audio/mpeg"`.
- 4.
5. **Not Providing Fallback Formats:** Relying exclusively on `.wav` files (which consume large amounts of bandwidth) instead of compressed `.mp3` or `.ogg` files.
- 6.
7. **Placing Text Outside the Fallback Area:** Expecting fallback text to show as a label. Text inside `<audio>...</audio>` is only rendered if the browser does not support HTML5.
- 8.

## Exam Definition

**HTML5 `<audio>` Tag:** A standard block/inline multimedia element introduced in HTML5 that embeds sound streams and audio files directly into an HTML document. It provides native player controls via the `controls` attribute and supports multiple source fallbacks using nested `<source>` tags with explicit MIME types.

## Quick Revision

- **Main Element:** `<audio> ... </audio>`
- 
- **Multi-Source Fallback:** `<source src="audio.mp3" type="audio/mpeg">`
- 
- **Key Attributes:**
- 
- `controls`: Displays the native player UI.
- 
- `autoplay`: Starts playback on page load (subject to browser policies).
- 
- `muted`: Silences audio output.

- 
- **loop**: Repeats audio continuously.
- 
- **preload**: Configures loading strategy (**auto**, **metadata**, **none**).
- 
- **Primary MIME Types**: **audio/mpeg** (MP3), **audio/ogg** (OGG), **audio/wav** (WAV).

# Embedding YouTube Videos in HTML

## Definition

Embedding a YouTube video in HTML means displaying and playing a video hosted on YouTube directly on your webpage. This is achieved using the HTML **<iframe>** (Inline Frame) element, which acts as a window to host another HTML document or media player inside the current page.

## Syntax

### Basic Syntax

#### HTML

```

iframe

width "560"

height "315"

src "https://www.youtube.com/embed/VIDEO_ID"

title "YouTube video player"

allow "accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"

allowfullscreen

iframe

```

- **<iframe>**: The container tag that loads external content.
- 
- **src**: The specific YouTube embed URL containing the unique video ID.
- 
- **title**: A text description of the frame content (mandatory for accessibility).
- 
- **allow**: Specifies feature policies and permissions granted to the iframe (e.g., **fullscreen**, **autoplay**).
-

- `allowfullscreen`: Allows the user to expand the video to full screen.
- 

## Explanation

### 1. The Video ID and URL Structure

A common beginner mistake is using a regular YouTube browser link inside an iframe. Standard watch links are blocked from running inside frames by YouTube's security policy (`X-Frame-Options`). You must convert the link to an embed link.

Link Type	URL Format	Works in <code>&lt;iframe&gt;?</code>
Standard Watch URL	<code>[https://www.youtube.com/watch?v=dQw4w9WgXcQ](https://www.youtube.com/watch?v=dQw4w9WgXcQ)</code>	<b>✗ No</b> (Refused to connect)
Short Share URL	<code>[https://youtu.be/dQw4w9WgXcQ](https://youtu.be/dQw4w9WgXcQ)</code>	<b>✗ No</b>
Official Embed URL	<code>[https://www.youtube.com/embed/dQw4w9WgXcQ](https://www.youtube.com/embed/dQw4w9WgXcQ)</code>	<b>Yes</b>

### 2. Key Attributes of the `<iframe>` Element

Attribute	Value	Description
<code>src</code>	URL	Points to <code>[https://www.youtube.com/embed/VIDEO_ID](https://www.youtube.com/embed/VIDEO_ID)</code> .
<code>title</code>	Text	Describes the frame for screen readers and search engines.

<code>width / height</code>	<b>Pixels</b>	<b>Sets the dimensions of the embedded player (Standard aspect ratio is 16:9).</b>
<code>allow</code>	<b>Policy list</b>	<b>Enables specific browser features like autoplay, gyro sensors, or picture-in-picture.</b>
<code>allowfullscreen</code>	<b>Boolean</b>	<b>Enables the player's native full-screen button.</b>
<code>loading</code>	<code>lazy / eager</code>	<code>loading="lazy"</code> delays loading the video until the user scrolls near it, speeding up page load.

### 3. YouTube Embed URL Parameters

You can customize the behavior of the video player by adding query parameters to the end of the `src` URL, starting with a question mark `?` and separating multiple parameters with an ampersand `&`.

Parameter	Values	Purpose	Example
<code>autoplay</code>	0 or 1	1 starts the video automatically (requires <code>mute=1</code> ).	<code>?autoplay=1&amp;mute=1</code>
<code>mute</code>	0 or 1	1 mutes the video audio by default.	<code>?mute=1</code>
<code>controls</code>	0 or 1	0 hides player controls like pause and volume. Default is 1.	<code>?controls=0</code>

<code>loop</code>	0 or 1	1 replays video infinitely (must be used with <code>playlist=VIDEO_ID</code> ).	<code>?loop=1&amp;playlist=VIDEO_ID</code>
<code>start</code>	Seconds	Starts playback at a specific timestamp.	<code>?start=45</code> (starts at 45s)
<code>end</code>	Seconds	Stops playback at a specific timestamp.	<code>?end=120</code> (stops at 2 min)

## Examples

### Example 1: Basic Video with Lazy Loading & Accessibility

#### HTML

<!-- Standard embed with lazy loading enabled -->

`iframe`

`width "560"`

`height "315"`

`src "https://www.youtube.com/embed/M7lc1UVf-VE"`

`title "YouTube video player demonstrating API features"`

`loading "lazy"`

`allow "accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"`

`allowfullscreen`

`iframe`

### Example 2: Autoplaying, Muted, and Looped Video

#### HTML

<!-- Autoplay works across modern browsers only when mute=1 is provided -->

`iframe`

`width "560"`

height "315"

src "https://www.youtube.com/embed/M7lc1UVf-VE?autoplay=1&mute=1&loop=1&playlist=M7lc1UVf-VE"

title "Autoplaying looping video demonstration"

allow "autoplay; encrypted-media"

allowfullscreen

iframe

## Practical Program

Save this code in an `index.html` file to test both fixed-dimension and modern responsive 16:9 YouTube video embeds:

### HTML

```
<!DOCTYPE html>
```

```
html lang "en"
```

```
head
```

```
meta charset "UTF-8"
```

```
meta name "viewport" content "width=device-width, initial-scale=1.0"
```

```
title
```

```
style
```

```
body
```

```
#f4f6f8
```

```
0
```

```
20px
```

```
#333
```

```
.container
```

```
800px
```

```
0
#ffffff
25px
8px
0 4px 10px rgba(0,0,0,0.1)

h1
#cc0000 /* YouTube Red Accent */
0

/* Modern Responsive Video Wrapper (16:9 Aspect Ratio) */
.video-container
100%
16 9
20px 0
8px
#000

.video-container iframe
100%
100%

.details-box
#eef2f7
```

```

 15px
 4px #cc0000
 15px
 0.95rem

style
head
body

div class "container"
 h1
 h1
 p
 p

<!-- Responsive YouTube Embed Container -->
div class "video-container"
 iframe
 src "https://www.youtube.com/embed/dQw4w9WgXcQ?start=5&mute=1"
 title "Sample Demonstration Video"
 loading "lazy"
 allow "accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"
 allowfullscreen
 iframe
div

div class "details-box"
 strong
 strong
```

```

 ul
 li strong strong code li
 li strong strong code code
 li strong strong code code
 li strong strong code
 code li
 ul
 div
 div

body
html

```

## Important Points

- **Hosting Advantage:** Embedding videos via YouTube saves your web server bandwidth and storage because Google handles the video streaming, encoding, and resolution scaling.
- 
- **Accessibility Requirement:** The W3C Web Content Accessibility Guidelines (WCAG) mandate that every `<iframe>` must include a meaningful `title` attribute explaining what content is inside the frame.
- 
- **Deprecated Attributes:** In older HTML versions, `frameborder="0"` and `scrolling="no"` were used. In HTML5, these attributes are obsolete; use CSS (`border: 0;` or `border: none;`) instead.
- 
- **Responsive Scaling:** To prevent the video player from overflowing small mobile screens, do not rely solely on fixed pixel dimensions like `width="560"`. Use CSS `width: 100%;` and `aspect-ratio: 16 / 9;`.
- 

## Common Mistakes

### 1. Using Watch URLs: Writing

`src="["https://www.youtube.com/watch?v=abc123xyz](https://www.youtube.com/watch?v=abc123xyz)".` This triggers a Refused to display in a frame because it set 'X-Frame-Options' to 'sameorigin' error in the browser console.

- 2.
3. **Missing `allowfullscreen`:** Leaving out this attribute disables the maximize button on the YouTube player.
- 4.
5. **Attempting Autoplay with Audio:** Adding `?autoplay=1` without `&mute=1`. Modern web browsers will automatically block the autoplay request.
- 6.
7. **Incorrect Looping Syntax:** Setting `?loop=1` alone without passing `&playlist=VIDEO_ID`. YouTube requires the `playlist` parameter pointing to the same video ID for single-video looping.
- 8.

## Exam Definition

YouTube Embed (`<iframe>`): An HTML implementation using the inline frame element (`<iframe>`) to embed third-party hosted YouTube video streams into a web document. It uses the destination URL pattern `[https://www.youtube.com/embed/VIDEO_ID](https://www.youtube.com/embed/VIDEO_ID)` and accepts query parameters for playback customization, allowing media delivery without consuming local server bandwidth.

## Quick Revision

- Core Tag: `<iframe> ... </iframe>`
- 
- Embed URL Pattern:  
`[https://www.youtube.com/embed/VIDEO_ID](https://www.youtube.com/embed/VIDEO_ID)`
- 
- Key Attributes: `src`, `title` (required for accessibility), `allow`, `allowfullscreen`, `loading="lazy"`.
- 
- Top Parameters:
  - 
  - `autoplay=1&mute=1` (Auto-play without sound)
  - 
  - `controls=0` (Hide controls)
  - 
  - `start=30` (Jump to 30 seconds)
  - 
  - `loop=1&playlist=VIDEO_ID` (Infinite loop)

